

SoK: A Defense-Oriented Evaluation of Software Supply Chain Security

Eman Abu Ishgair
Electrical and Computer Engineering
Purdue University
West Lafayette, USA
eabuishg@purdue.edu

Juanita Gomez
Computer Science and Engineering
University of California, Santa Cruz
Santa Cruz, USA
jgomez91@ucsc.edu

Marcela S. Melara
Corporate Technology Office
Intel Corporation
Hillsboro, USA
marcela.melara@intel.com

Alvaro Cardenas
Computer Science and Engineering
University of California, Santa Cruz
Santa Cruz, USA
alacarde@ucsc.edu

Santiago Torres-Arias
Electrical and Computer Engineering
Purdue University
West Lafayette, USA
santiagotorres@purdue.edu

Abstract—The software supply chain is a highly complex ecosystem of tools, workflows, and human factors involved in creating a piece of software. Due to its central role in critical systems, it has become a frequent target of sophisticated attacks. Prior work has focused primarily on classifying attack vectors, but there is a gap in discussing and understanding effective defenses. This paper proposes a defense-oriented systematization of software supply chain security that models the fundamental components and causal relationships. Based on 76 works from academia and industry, we define key security objectives and defense patterns of established software supply chain defense techniques. The goal of our taxonomy is to identify major opportunities for future research, and aid software supply chain practitioners when reasoning about the effectiveness of defenses available today.

Index Terms—Supply chain security, Computer security, Risk analysis, Software engineering, Data models, Authentication, Integrity, Provenance

1. Introduction

Recent high-profile attacks on the software supply chain (e.g., [23, 42, 45, 55, 66, 79, 142]) have motivated governments [39, 165, 193] and industry associations to develop frameworks to audit the Software Development Life Cycle (SDLC) and validate open source and proprietary software (e.g., [26, 81, 83, 110, 129, 133, 134, 150, 154, 157]).

Although the research community has developed detailed taxonomies of open-source software supply chain attacks (e.g. [32, 49, 69, 92, 95, 98, 99, 130, 198]), this adversary-centric perspective leaves practitioners with two key challenges. First, these taxonomies rarely specify the *security objectives* defenders should pursue to counter identified threats. Second, they focus on isolated stages of the supply chain, such as package repositories or CI pipelines, ignoring the *transitive and interconnected nature of modern ecosystems*, where a single weakness can cascade throughout the chain.

As a result, the defense landscape is fragmented: administrators scramble to adopt point solutions (SBOM generators, secret scanners, sandboxed builders, etc.) without clear guidance on how these tools interoperate or where critical gaps remain.

In this paper, we focus on a defense-oriented systematization of software supply chain security. We introduce **AStRA**, a graph-based model that treats a supply chain as a directed acyclic graph of **principals**, **Steps**, **Resources** and **Artifacts**. We then perform a systematization of software supply chain defenses from academic papers and the gray literature, classifying each defense by *what* it protects, *how* it works, and *where* it fits in the supply chain DAG. Our bottom-up analysis reveals hidden interdependencies between defenses and allows us to derive a set of fundamental *security objectives* whose aim is to contain most documented software supply chain attacks. Further, we highlight objectives with limited coverage and map them to cross-cutting *defense patterns* that describe how a defense technique meets its security objective.

As such, our paper builds on methods in [132] and complements attack-oriented work [98, 130] by providing an objective-driven framework to understand and apply targeted defenses throughout the software supply chain.

Positioning Relative to Prior Taxonomies. While attack-oriented taxonomies provide valuable insight into how systems are compromised by classifying adversary techniques and vectors, they do not explicitly define the security properties that defenses must guarantee, leaving practitioners to infer what protections are required.

In contrast, our work takes a defense-oriented approach by deriving a set of security objectives that specify the guarantees a secure supply chain must satisfy, independent of specific attacks. This enables reasoning not only about individual mechanisms, but also about coverage and gaps across the supply chain as a whole.

Prior work has explored related ideas in limited contexts. For example, defining workflow-specific objectives [93] or enumerating safeguards [98], but these

efforts either focus on particular environments (e.g., CI systems) or catalog mechanisms rather than formalizing cross-cutting objectives. As a result, guidance in practice is often expressed as collections of best practices (e.g., signing, MFA, sandboxing), without clearly specifying the security guarantees they are intended to provide. We instead distinguish between mechanisms and objectives: best practices (e.g., signing, MFA, sandboxing) are mechanisms that describes how to defend, independent of implementation, whereas security objectives define properties that must hold (e.g., non-equivocation). Best practices are often technology- and implementation-specific and may evolve as new tools emerge. In contrast, security objectives capture guarantees that can be realized through different defense mechanisms over time. This abstraction also allows for flexibility when choosing techniques based on operational context if they satisfy the same underlying guarantees.

Our main contributions include:

- **AStRA, a principled model of the software supply chain.** We formalize four core elements and their causal relationships, allowing more precise representation of and reasoning about transitive risk.
- **An objective-driven defense taxonomy.** We derive fifteen foundational security objectives through systematic evaluation of prevalent academic and industrial mechanisms, and their properties.
- **A unified systematization of defenses.** Our analysis spans 76 techniques (e.g., transparency logs, threshold signing) and specialized tools (e.g., container isolation, provenance attestations), revealing both overlaps and blind spots.
- **Actionable insights and research directions.** We identify underexplored areas, such as DAG-level integrity, usable signing workflows, artifact confidentiality, and AI-generated code provenance, and outline concrete opportunities for the community to close these gaps.
- **New scripts and datasets.** We also make publicly available the scripts we used to collect the papers used in our systematization and our collected results <https://github.com/TSELab/astra-artifacts>

2. Systematization Methodology

Our methodology consists of two primary phases: 1) automated collection of academic databases using keywords and a list of venues, and 2) a multiple-step review and labeling process by at least two coauthors of this paper.

2.1. Phase 1: Paper collection

Following guidelines for systematic literature studies [184], we collect a seed of papers using keyword search and then use this seed to further expand the corpus of work to study. To collect as much relevant work as possible in this first phase, we filter the work in the second phase, so we look for the term "supply chain" and variations of it like "supply-chain" in security conferences.

To ensure high quality and impactful research, and given that an open query would result in an unmanageable number of papers (More than 1,850,000 results as of 2025), we constrain our search to publications within established computer security venues. We seed our search with system security conferences with large citation rates (citations/accepted papers), participant rates (registered participants/accepted papers), and low acceptance rates, as summarized in [199]. We list the selected conferences in Table B in App. B. We then use Google Scholar's API and search for the conference sources using keywords such as "supply chain" `source:"USENIX Security Symposium"` as a query to filter supply chain work presented at the USENIX Security Symposium. For each paper, we extract the title, authors, year, venue, abstract, citations, and URL. The results of this search, conducted between February and June 2025, are compiled into a single dataset of 671 potential software supply chain security papers. The collected papers can be found in our shared repository <https://github.com/anonymous-eurosp/astra-artifacts>.

An interesting anecdote from these results is that we found citations that appear to be hallucinated by an AI agent. In particular, Google Scholar has entries with "supply chain" in the title but has no direct link to the proceedings of the conference, and when we looked at the proceedings of that year, we did not find the collected publication.

2.2. Phase 2: Manual Filtering and Scope

Phase 1 resulted in more than 100 papers on the AI supply chain and the hardware supply chain. We deliberately scope this study to the software supply chain to provide a focused systematization of the defenses within this rapidly evolving and practically impactful domain. Although related areas such as the AI supply chain and the hardware supply chain raise equally important security concerns, including data poisoning, model tampering, or hardware trojans, each involves distinct threat models, stakeholders, lifecycle processes, and technical challenges. These domains merit their own in-depth systematization efforts to account for their unique complexities and to avoid conflating fundamentally different ecosystems under a single framework. Our goal is to provide clarity and structure to the software supply chain security landscape while encouraging future but complementary SoK work that can do the same for AI and hardware supply chain defenses.

Therefore, we perform various filtering passes, ensuring that each paper is reviewed by at least two authors who agree on the following questions:

- 1) Does the paper focus on the software supply chain, rather than physical, hardware, or AI supply chains?
- 2) Is software supply chain security a central topic of the paper, rather than a peripheral motivation (e.g., a security challenge) or a side note?

Disagreements between the reviewers were resolved through discussion, with a third coauthor serving as tie-breaker if needed to ensure consistency and minimize

bias. Of the 671 papers from Phase 1, 98 of them are marked as “relevant” by both reviewers. We then classify these papers according to their core contribution to supply chain security. We identify four main categories as described below (see Table 1).

- **Attack:** If the primary contribution of the paper is attacks on software supply chains.
- **Defense:** If the main contribution of the paper is a new or newly applied defense mechanism.
- **Analysis:** If the paper evaluates the effectiveness of *existing* security controls, policies, or practices within the supply chain by conducting a measurement study (e.g., the extent to which Android applications update their third-party libraries) or analyzing the security of existing defenses (e.g., using formal methods to evaluate a defense).
- **Survey:** If the paper provides a review, taxonomy, or Systematization of Knowledge for existing work in the area of software supply chain security.

TABLE 1. CATEGORIZED ACADEMIC REFERENCES (FILTERED + EXTENDED).

Category	References	Count
Defense	[14, 18, 29, 30, 37, 41, 50, 51, 54, 64, 65, 69, 73, 75, 77, 78, 84, 85, 88, 100, 106, 107, 112, 113, 115, 117, 119, 121, 122, 123, 128, 136, 147, 155, 160, 168, 174, 175, 179, 181, 185, 188, 189, 191, 192, 200]	46
Attack	[19, 28, 31, 40, 70, 130, 143, 180, 194]	9
Security Analysis	[20, 22, 34, 44, 46, 48, 57, 58, 71, 72, 76, 86, 87, 89, 90, 91, 92, 94, 101, 102, 103, 104, 105, 108, 109, 120, 124, 135, 137, 138, 139, 140, 148, 158, 159, 178, 182, 183, 196, 197, 198]	41
Survey	[74, 98]	2

Extending paper selection. We select the 46 defense papers identified in phase 2, and enrich our paper selection by consulting industry forums, guidelines, open source recommendations, and government requirements. However, a systematic aggregator of gray literature is not available, so we rely on the author’s engagement with industry and government to select adequate texts. Specifically, we selected work from the Internet Engineering Task Force (IETF) [26], the U.S. National Institute of Standards and Technology (NIST) [33], the Center for Internet Security (CIS) [114], the Linux Foundation [38], and large open source communities [25, 96]. We also extend our selection of papers to include proposed systems (these may also be published in top systems and networking conferences) if they are mentioned in these guidelines. Similarly, we identify tools and frameworks that instantiate properties outlined in these guidelines. This adds 6 papers from industry literature, and 24 tools that were not introduced via academic work, yielding a total of 76 defense-oriented works. The table containing these additional works can be found in Table 4 in the appendix.

As the field of software supply chain research has matured, we can see that surveys and SoKs started in 2023. As mentioned above, a previous SoK focused on attacks [98], and a more recent one [74] focuses only on docker attacks and defenses. Our goal is to complement these previous efforts by focusing on defenses against

a wide range of supply chain threats. We elaborate on trends in the distribution of software supply chain research references in App. B.

3. Modeling the Software Supply Chain

The literature surveyed tells us that modern software supply chains span numerous actors, tools, services, and artifacts distributed across organizational and administrative boundaries. Understanding the security of how software is created, transformed, validated, and released requires more than simply identifying individual vulnerabilities; it requires evaluating defenses, reasoning about transitive risk, diagnosing compromises, and understanding how different components interact.

Specifically, stakeholders ask software supply chain security questions such as: Which elements influence a given artifact? Where are trust boundaries crossed? How does a compromise in one part of the ecosystem propagate to others? And how do different defenses address key security goals?

Addressing these questions requires a precise model of the structure and behavior of the supply chain. Graph-based representations help capture the relationships among code dependencies, build steps, and human actors, enabling the analysis of transitive risks and potential attack paths. Two such representations dominate the literature: dependency graphs and flow graphs.

Dependency graphs map the relationships between software components based on reliance and consumption at a single point in time t . In a dependency graph, the nodes represent packages or components, and the edges represent “artifact adjacency”, that is, one artifact depends on or includes another [32, 46, 49, 120, 180, 198, 201]. Of note, dependency graphs are *speculative*, for most dependencies are not *resolved* until an artifact propagates through the chain. A good example is package dependency networks, which rely on a package manager to carry out *dependency resolution* to identify the best candidates at the current time t .

In contrast, *flow graphs* focus on representing the sequence of transformation steps that result in the creation of a software artifact – often likened to a CI/CD pipeline or development lifecycle [71, 80, 116, 117, 169]. Here, nodes represent build steps, processes, or stages (such as compile, test, package, deploy), and directed edges represent the sequence or data flow between these stages.

However, these abstract models are insufficient for reasoning about supply chain security because they fail to capture how actions taken by one element *influence* the next. Consider an attack on a CI/CD pipeline where a compromised build runner injects malicious code during compilation. A dependency graph cannot represent this scenario: artifact adjacency remains unchanged – the same package depends on the same libraries – yet the resulting artifact is malicious. A flow graph performs only slightly better: it captures the “build step” but cannot express which resource carried out the step, which principal authorized it, or why the resulting artifact deviated from expectations.

Both models lack the notion of *causality*: i.e., they cannot represent that a specific action, performed by a specific actor using a specific resource, directly or *indirectly*, caused a compromise. Without reasoning about causality, neither model can explain how a compromise propagates: whether an attack originated from a poisoned dependency, a misconfigured CI service, a stolen credential, or a removed validation step. As shown in other domains, such as root-cause analysis for intrusion detection [68], this missing causal structure is precisely what prevents these models from supporting post-mortem analysis, transitive risk reasoning, and principled defense evaluation.

3.1. A Causal Supply Chain Model

To address these limitations, we introduce a new causality model for the supply chain, which we call AStRA. AStRA treats the entire software supply chain as a directed acyclic graph (DAG) of four element types: **Principals** (e.g., people or organizations), **Artifacts** (e.g., code or artifacts produced/consumed), **Resources** (e.g., tools, services, infrastructure), and **Steps** (e.g., actions or operations). Edges in this graph represent causal links: Principals use Resources; Resources carry out Steps; and Steps consume input Artifacts and produce output Artifacts. This forms an actor-artifact graph that richly encodes who did an action with what resource, leading to what artifact.

This causal graph expands traditional dependency graphs, which only show artifact-to-artifact relationships, and beyond flow graphs, which show step sequences without tying in the human or resource responsible for an action. By including actors and distinguishing “active” nodes (such as build systems, signing services, compilers) from “passive” nodes (code, binaries, configurations), AStRA can represent the context relevant to security that earlier models miss. For instance, in-toto’s [168] layout model records steps and their materials, but does not explicitly differentiate whether a given input was a piece of source code or a compiler program – AStRA makes that distinction clear by labeling the latter as a Resource.

When evaluating the design and effectiveness of software supply chain defenses, understanding how a malicious artifact, for instance, can be introduced is essential (via a compromised dependency artifact or via a compromised build tool resource?). Thus, we leverage AStRA to systematize software supply chain defenses in a way that makes causality explicit. This model will also help us systematize the defenses.

AStRA example. Figure 1 shows a software supply chain DAG structure that covers three common steps:

Source is the initial phase, which involves the creation and editing of the source code for the final artifact. This code is typically stored and managed in a repository hosted on platforms such as GitHub [11].

Build refers to the process of transforming input artifacts (source code) into output artifacts (e.g., a binary). The resources used to carry out this step often consist of a developer’s local machine or a cloud-based CI/CD service.

Package is often the final step in which the output of a build process is “packaged” to facilitate “publishing” the final artifact for use by others. Package artifacts can be found in ecosystem-specific package managers like PyPI (Python Package Index) or NPM (Node Package Manager).

Note that more complicated structures often exist and may include steps to validate the behavior of an artifact (e.g., by means of running a test suite on a binary), or perform security-related tasks (e.g., code signing). However, all steps carried out in the chain can be represented as a specific DAG structure of the elements introduced above.

To illustrate the practical application of our framework, we use the scenario in Figure 1 as a running example throughout this paper. This scenario represents a Containerized Microservice Build Pipeline. In this context, the Principal is represented by a Developer (Alice), the Steps include ‘Compile’ and ‘Containerize’, the Resources consist of the GitHub Actions Runner, and the resulting Artifacts are the Source Code repository and final Docker Image.

We validate AStRA’s fidelity and generality in Appendix A (as well as showing how AStRA can be used to model popular supply chain failures). In particular we show an application of our tool to model the supply chain for 990 debian builds in Figure 5.

3.2. Threat Models

By formalizing these four elements, we can precisely define the threat landscape. We summarize the threat model for each element, plus the DAG structure itself:

Threats to *principals* aim to subvert the authentication and authorization process by stealing or reusing credentials [130], or by exploiting overly privileged roles [92]. A compromised principal can authorize malicious steps downstream: such as approval of a tampered build [141], altering a critical artifact [66], or modifying CI logic while remaining indistinguishable from a legitimate principal [120].

Artifacts are targets for tampering during creation [28], in transit [32], or maliciously substituted after caching or redistribution [67]. These attacks exploit the fact that artifacts are frequently reused across packages and systems.

Threats to *resources* seek to change a resource’s intended behavior during a step, or abuse over-privileged execution environments to perform unauthorized actions [70]. Because resources are often reused across different steps and supply chains, compromising them can have wide reaching effects across the supply chain graph [171].

Threats to *steps* aim to modify the behavior of a step to operate in an unexpected or unauthorized way like running malicious builds [130, 141] or to conceal attacks on related supply chain elements. Manipulated step parameters (e.g., altering configuration flags to disable security checks [79]) is a common threat. Attackers can also compromise a step via any other element, e.g., using a compromised compiler resource [10].

Finally, threats to the *supply chain DAG* aim to add or remove vertices or edges to alter the flow and

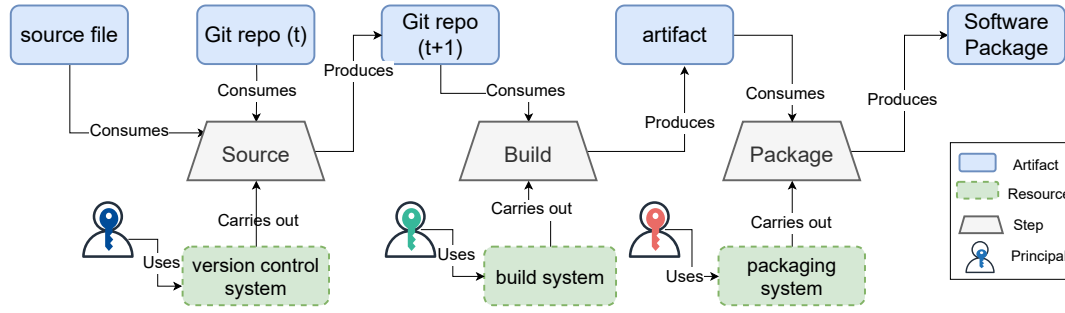


Figure 1. Software supply chain DAG structure example that consist of three standard steps: sourcing, building, and packaging.

outcome of a software supply chain. Real-world examples include removing an artifact from a package manager [127], or adding a malicious package during the creation of a Docker image [27]. Another form of structural manipulation omits critical steps. For example, misconfigured GitHub Actions workflows allow untrusted pull requests to trigger a publish step without validation (e.g., tests or reviews) [92], enabling attackers to propagate malicious artifacts.

Insight 1: Each software supply chain element (incl. the entire DAG structure) presents an attack vector with a unique threat model. While threats target standard security expectations (i.e., confidentiality, integrity, or availability) of the different elements, practitioners and researchers lack a unified framework for reasoning about the effectiveness of defenses against these threats.

4. Defense Systematization

We organize the defenses in our corpus using AS-TRA (§3), categorizing each technique by the security objective it addresses and the supply-chain element it protects. This structure allows us to compare defenses across objectives and identify recurring patterns that reveal both overlaps and gaps in the current landscape. Table 2 summarizes the four dimensions of our taxonomy.

- **Defense Technique:** The specific technical mechanism or approach used in the defense; techniques are often well-established in other areas of security, but newer to the supply chain.
- **Security Objective:** The primary security goal that a defense technique is meant to achieve.
- **AS-TRA Element:** One of five classes of participants in the software supply chain (§3) that a defense targets, i.e., principals, artifacts, steps, resources and the overall DAG structure.
- **Defense Pattern:** How the defense implements a particular security objective based on established security control types [187].

To organize our systematization, we enumerate security objectives for each AS-TRA element: **P#** for principals (§4.1), **A#** for artifacts (§4.2), **R#** for resources (§4.3), **S#** for steps (§4.4), and **T#** for the supply chain DAG structure (§4.5). Under each security objective, we

then discuss existing defense techniques that achieve the objective and their effectiveness. The order in which we discuss the elements is based on our insight that several approaches build on top of defenses for the preceding elements (our order also follows the rows in Table 2); for example, as resources encompass both tool artifacts and services, defenses for resources builds upon artifact defenses.

4.1. Defending Principals

As in other systems, supply chain security begins with principal *authentication* and *authorization*. The defenses we observe are focused on protecting, detecting, and reducing authentication and authorization failures.

To ground these concepts, we utilize the running example from Figure 1, where a Developer (Alice) acts as the primary Principal. Throughout this section, we illustrate how these defense patterns apply to Alice's interactions with the build system to mitigate unauthorized access and tampering.

P1 - Credential reuse limited. Limiting the validity of a credential by time or use reduces how long an attacker can exploit it.

Ephemeral credentials constrain the window of compromise by limiting the time frame during which a credential is valid [38]. The most common form used in the software supply chain today are short-lived certificates [167], digital certificates used to authenticate a remote party that have a short validity period, i.e., they expire after a short period of time (e.g., a few minutes [153] to a few days [167]). For instance, Sigstore's Fulcio [150, 151] provides a framework for issuing short-lived X.509 certificates for code signing to curb the impact of compromised software distribution.

P2 - Credential compromise detected. A key objective for securing principals is detecting whether credentials have been leaked, replaced, or misused.

Secret scanning analyzes resources searching for unintentionally exposed credentials and cryptographic secrets [38]. Trufflehog [170], for example, is capable of identifying different types of leaked secrets on resources like git repositories and local file systems. Key-Sentinel [200] improves secret scanning accuracy using machine learning and semantic analysis.

Credential transparency typically relies on a database backed by a Merkle tree referred to as a *trans-*

TABLE 2. SUPPLY CHAIN DEFENSE TECHNIQUES CATEGORIZED BY ASTRa ELEMENTS, SECURITY OBJECTIVES AND DEFENSE PATTERNS. ● DENOTES A TECHNIQUE THAT MEETS AN OBJECTIVE AND FALLS UNDER A DEFENSE PATTERN. ○ DENOTES CONDITIONAL/LIMITED FULFILLMENT. △ DENOTES A POTENTIAL CONFLICT WITH OTHER OBJECTIVES. ✓ DENOTES A DEFENSE TECHNIQUE THAT WAS DEVELOPED SPECIFICALLY FOR THE SOFTWARE SUPPLY CHAIN. TABLE 5 IN APPENDIX C CONTAINS OUR COMPLETE CORPUS.

AStra Element	Supply Chain Specific?	Academic Examples	Industry Examples	Security Objective	Defense Pattern					
					Detective	Corrective	Preventative			
					Transparency	Integrity	Graceful degra.	Redundancy	Limitation	Correctness
Principals ephemeral credentials secret scanning credential transparency role separation multi-factor authentication separation of privilege threshold authorization		[121] [200] [121] [92]	[1] [62, 126] [195] [38]	P1 P2 P3 P4 ● ● ● ● ● ● ● ●	● ● ● ● ● ● ● ●	● ● ● ● ● ● ● ●	● ● ● ● ● ● ● ●	● ● ● ● ● ● ● ●	● ● ● ● ● ● ● ●	● ● ● ● ● ● ● ●
Artifacts hashing artifact signing metadata signing artifact transparency ledger-based systems static analysis patch propagation dynamic analysis ML-based analysis	✓ ✓ ✓	[175] [121] [88] [113, 121] [18] [77, 78, 100] [50, 106, 107] [75, 136, 174] [84, 85, 147]	[60, 61] [47] [133, 150] [162]	A1 A2 A3 ●						

parency log, which makes any changes to the contents of the log easily detectable. Sigstore [150] implements this approach for the software supply chain by providing a specification and framework to leverage certificate transparency [1] and audit the validity of Fulcio-issued code signing certificates. Some implementations [121] also preserve some security properties so long as there is one honest principal, providing additional reliability in the face of compromise (see P4).

P3 - Credential authority limited. Several techniques restrict the scope of a credential to increase the effort required to abuse a compromised principal.

Role separation limits each credential to a defined set of tasks by grouping principals according to their function in the supply chain [33, 38, 92]. By minimizing shared authority, it reduces opportunities for interference and enforces least privilege.

Platforms such as GitHub implement role-based repository permissions [62], and package managers such as npm apply similar role models [126].

Multi-factor authentication (MFA) requires principals to authenticate using at least two credential categories. Following industry software supply chain best practices [38], major package managers [125] and

version control systems [63] adopted two-factor authentication.

Most MFA integrations [63, 125] in the supply chain leverage one-time passwords (OTPs) and physical keys (e.g., Yubikeys [195]) to limit credential reuse (P1).

P4 - Multiple principals required. To limit the impact of a compromised or malicious principal, some defenses require that sensitive tasks be authorized or executed by multiple independent principals.

Separation of principal privilege requires more than one principal to authorize a step, preventing any single actor from holding full authority. A common industry-recommended practice [38] is two-person code review, in which at least two qualified reviewers must approve changes before they are merged. This reduces the risk of undetected issues and mitigates malicious contributions from malicious principals.

Threshold authorization divides the credential or signing capability for a step across n principals, requiring a quorum to complete the operation. This not only raises the bar for unauthorized access, but also limits the authority of any single credential (P3).

The in-toto framework [168], implements threshold models to maintain integrity even if some principals

are compromised. Sigstore [121] similarly recommends threshold signing to reduce the likelihood that a compromised key can be used to release tampered artifacts.

Insight 2: Protecting principal authentication and authorization is the first line of defense for the software supply chain. A combination of principal defense techniques seems necessary to significantly enhance overall supply chain security, with practical implementations available to meet most security objectives.

4.2. Defending Artifacts

Artifacts are the unit of exchange in the software supply chain. Attacks focus on surreptitiously modifying or replacing an authentic artifact with a compromised one.

In our running example Figure 1, the primary Artifacts are the Source Code and the final Docker Image. As the Source Code is transformed by the Build Step, defense patterns are applied to ensure that the resulting Docker Image accurately reflects the audited source and that its provenance is cryptographically verifiable by downstream consumers.

A1 - Artifact changes detected. A core objective is detecting tampering with artifacts at rest, in transit, or in memory.

Hashing uses cryptographically strong hash functions (e.g., SHA-256) to detect bit-level changes. Because collisions are extremely unlikely, hashing enables artifact identification and content matching [33, 38, 156] in settings such as source code [59] and package repositories [43]. However, hashing alone only partially satisfies A1 [17, 169]. If an attacker can replace a published artifact, they can typically replace its checksum as well. For instance, an attacker who can replace a published artifact on a website is likely able to also replace the checksum. In fact, given that these checksums are distributed through the same channel as the software artifact, any compromise of the channel subverts the integrity guarantees of hashing.

Artifact signing generates a digital signature over the contents of a software artifact (e.g., a package). A range of algorithms are used in practice: Sigstore [121] and Microsoft Authenticode [163] use x.509 certificates to sign OCI containers and device drivers, while Debian and Maven rely on GPG signing [43, 56]. Version-controlled code, such as GitHub repositories, also commonly uses GPG signatures [60, 61]. Because signature verification relies on trust metadata delivered through a separate channel, signing mitigates many of the weaknesses of hashing. Still, tampering with metadata can circumvent integrity guarantees even for signed artifacts [32, 169], meaning artifact signing only partially fulfills A1.

Metadata signing extends these protections to metadata (e.g., file creation time), preventing attackers from manipulating metadata as an additional attack vector. A widely deployed implementation is The Update Framework (TUF) [145], which signs package repository metadata and uses role separation (see §4.1) and publicly visible “freshness claims” [145] to provide

a degree of artifact non-equivocation (see A2). Other variants of TUF exist for IoT and Automotive, notably, Uptane [88].

Similarly, signing Docker container manifests [47] transitively signs both the manifest and its associated metadata (e.g., layers, size, checksum). Combined with artifact signing, metadata signing can make unauthorized artifact changes fully detectable.

A2 - Artifact non-equivocation. This objective ensures that all principals observe the same artifact produced and consumed by a step.

Artifact transparency uses transparency logs to provide a verifiable history of an artifact’s changes over time. This makes artifact tampering and surreptitious or compromised releases detectable (A1), but the degree of transparency depends on the log’s deployment (local or public). Because credentials are a special case of artifacts, this mechanism naturally generalizes credential transparency (see §4.1).

Examples include Scorecards [133], which analyzes source repositories for security-related practices and assigns a security posture score [38]; Firefox’s Binary Transparency [5], which published signed binaries in a public log; and Sigstore’s Rekor log [152], which records a wide range of artifacts, including signatures themselves, in-toto attestations [3], software bills of materials, and other arbitrary binary blobs. Speranza [113] takes a privacy-preserving approach by using anonymous credentials and zero-knowledge proofs so that only authorized signers can produce valid signatures without revealing their identities.

However, transparency logs are vulnerable to fork attacks unless supplemented with additional monitoring mechanisms [161]. As a result, artifact transparency alone only partially satisfies A2.

Ledger-backed systems strengthen non-equivocation guarantees by using distributed ledgers (DLs) to make tampering and omissions *globally* visible. The ledger’s append-only structure and consensus protocol provide strong tamper-detection properties (A1). Contour [18], for example, leverages Bitcoin’s ledger to help users detect these attacks.

A3 - Expected artifact behavior enforced. Defenses under this objective ensure that artifacts behave as intended.

Static analysis is used to identify suspicious components such as installed packages (e.g., by detecting a vulnerable dependency), or vulnerabilities that may be introduced transitively (e.g., a statically compiled vulnerable library [38, 156]). At the core of these approaches lies a mechanism for evaluating artifact representations, such as an abstract syntax tree (AST) or binary code (e.g., [162]).

Some systems [95, 117] use static taint analysis to trace flows through artifacts such as PHP web applications and GitHub Actions workflows, identifying risky execution paths and misconfigurations. Others detect abnormal code behavior in specific language code (for example, [99, 147] focus on Java). CodeQL-based analysis [64] identifies malicious JavaScript patterns (e.g., suspicious API usage).

Static comparison of source code and corresponding published artifacts [175] can reveal unautho-

rized modifications introduced during build or release (A1). Recent work also explores vulnerability detection via feature classification of reused code components [185], and static analysis is increasingly embedded into DevSecOps workflows for automated risk enforcement [51].

Patch propagation builds on static analysis to generate and apply source code patches that propagate security patches to forked codebases [107]. Similarly, VFCFinder [50] and SPatch [106] use symbolic code execution and natural language processing, respectively, to identify, generate and apply patches to source code. Unlike direct static analysis techniques, patch propagation not only detects anomalous behavior but also aims to correct it.

Dynamic analysis FieldFuzz [30], 5G-Spector [181], and FV8 [136] dynamically analyze artifacts ranging from automation runtimes and cellular infrastructure to JavaScript packages, revealing threats in third-party components. Hybrid analyses combine static and dynamic signals: DECAF [37] removes unused firmware while preserving runtime correctness; Rapid-Patch validates automatically generated firmware hot-patches [75]; DONAPI [77] detects malicious npm packages by integrating static graphs with runtime traces; and [128] identifies subtle API misuse in third-party libraries.

Machine learning (ML) based analysis is an emerging line of work. Jarhead [147] combines static analysis and ML to classify Java packages as benign or malicious. Ladisa et al. [100] and Latch [189] apply tree-based models to detect malicious npm and Python packages.

Active Learning and Regeneration (ALR) systems such as HARP [174] and Up2Dep [122] detect abnormal application behavior—such as suspicious file system changes or global state manipulation—by modeling expected functionality in a controlled environment to avoid learning malicious patterns. These models can regenerate vulnerability-free versions of libraries in JavaScript and C/C++. Recent work [192] also uses large language models to learn semantic patterns that distinguish benign from malicious source code, even across varied coding styles and obfuscation techniques. Although dynamic and ML-based techniques can effectively enforce expected behavior, their ability to rewrite or regenerate code or binaries may conflict with the detection of artifact changes (A1).

4.3. Defending Resources

As described in §3, resources are themselves software components, i.e., they are created through their own software supply chains. This means that many security objectives that apply to resources may be met by artifact defenses (see §4.2). At the same time, because resources are used to carry out (parts of) a particular supply chain step, we identify additional security objectives specific to resources, which pertain to mitigating execution-time risks.

For a resource to operate, it typically requires auxiliary resources and artifacts, which we generally refer to as the *execution environment*. Depending on the type of resource, its execution environment may be defined

at various levels of granularity ranging from an intra-process unit (e.g., thread or domain), to an application, to a virtualized environment (e.g., container or VM), to an entire system running on a hardware platform. Thus, to reduce risks in resources we must protect different aspects of execution environments.

In our running example Figure 1, the primary Resource is the GitHub Actions Runner. By applying defense patterns to this resource, we ensure that the environment used to execute the 'Compile' step is authenticated and hasn't been tampered with before the process begins.

The Log4j vulnerability highlights the supply chain risks associated with widespread use of open-source dependencies as resources. In December 2021, a vulnerability was discovered in the log4j utility, which that enabled remote attackers to inject and remotely execute code through log messages. After its discovery, millions of attempted exploits, many of which turned into successful denial-of-service (DoS) attacks [171].

R1 - Resource privilege limited: Limits the operating privileges a resource is granted to only those needed to carry out a step *within* its execution environment. This objective is equivalent to P3 aiming to reduce risk within the context of an authorized step.

Mandatory Access Control (MAC) enforces a non-discretionary, global access policy which enforces multiple levels of security for operations resources perform [144]. As such, MAC can be used to meet objective (R1) as a compromised resource can be prevented from performing an unauthorized operation by default.

One way these defenses may help reduce software supply chain risk is by running a kernel-level MAC system like AppArmor [172] or SELinux [8] to control which operations (e.g., `read()`, `mount()`) a CI/CD or build resources may perform on artifacts [38].

R2 - Environment interfaces restricted: Restricts the access points to and from a resource's execution environment.

Filtering techniques intercept accesses at a particular environment interface controlling the level of interactions between internal and external resources. HODOR [179] applies combined static-dynamic call graph analysis to enforce thread-specific seccomp syscall filtering in Node.js, limiting resource privileges R1 and restricting environment interfaces R2, with partial enforcement of execution boundaries (R3). Defenses such as eBPF [52] provide programmable filtering of a variety of kernel-level interfaces (e.g., network, system calls), which also enables enforcement of MAC policies for specific resources running on a system (R1, R2). While eBPF [52] executes within an isolated environment within the kernel, we do not consider this sufficient to meet R3 because boundaries are only enforced around the defense mechanism itself. Nati-Sand [14] similarly applies eBPF and other techniques to sandbox native modules in JavaScript runtimes, enforcing syscall-level policies to restrict untrusted dependencies.

R3 - Environment boundaries enforced: Explicitly defines and enforces the boundaries of a resource's execution environment. This objective aims to prevent any

interference between different resource environments, even in the face of any fault or compromise.

Virtualization creates an abstract execution environment in which resources share common underlying hardware and software components without needing to be aware of other environments [53]. Software virtualization techniques establish boundaries based on specific common properties or conditions shared by resources that are typically enforced by privileged software (R3). Hardware virtualization approaches (e.g., [13, 173]), on the other hand, are designed to enforce strong boundaries and controlled I/O interfaces using dedicated CPU features (R2, R3). Virtual machines are very commonly used in the software supply chain to implement strongly isolated execution environments for resources such as CI/CD runners [69].

4.4. Defending Steps

Steps represent software development actions or operations that principals carry out on artifacts by use of resources. Generally, step risks depend on whether risks to principals, resources and artifacts in a step have been addressed. For example, compromising a principal who is authorized to carry out a step leads to a compromised step.

Yet, even with reduced risks to principals, artifacts and resources, steps may still face risks related to the invocation or operation of the step.

In our running example Figure 1, the primary Steps are 'Compile' and 'Containerize'. By applying defense patterns to these steps, we ensure that the transformation from Source Code to Docker Image is isolated from external interference and that the process itself produces verifiable evidence of its execution.

While most of the approaches we evaluate target build and packaging steps, their design principles and properties can be applied to other steps in the software supply chain. These mechanisms fall within two major categories: Preventative and detection approaches.

S1 - Expected step behavior enforced: The expected step behavior is well-defined and cannot be altered once triggered. Ensuring correct step behavior may be achieved by the correctness of the resource(s) used to carry out a step.

Deterministic steps follow patterns that prevent sources of entropy from influencing a steps' behavior or outcome (S1). For instance, Courtès et al. [41], describes how GUIX ensures correctness via source-to-binary mapping.

Navarro-Leija et al. proposed DetTrace [119], a reproducible container abstraction that enables reproducible software builds, among other use cases. To this end, DetTrace controls the influence of a system's behavior, such as naturally noisy system calls (e.g., `clock_gettime`) that would introduce nondeterminism into a step (S1).

Static analysis Other approaches apply static techniques to model authorship fingerprints in compiled binaries [112], match recurring vulnerability fixes via semantic equivalence [78]. Static analysis has also been applied detecting missing verification steps in firmware

update mechanisms [188]. Since static analysis techniques tend to be domain-specific and require a separate enforcement mechanism, however, this approach only partially enforces expected artifact behavior.

S2 - Step operation recorded: The identities of principals, artifacts used and produced, and resources used, are recorded. Further, a step provides faithful information about its operation.

Step transparency provides information about any aspect of a step's operation to provide visibility into all aspects of a step (S2). A prominent type of step metadata are Software Bills of Materials [164] for their use in software supply chain compliance [193]. Their main goal, while not authenticated, is to convey information about artifact composition, software "pedigree", and quality, complementing other operational metadata such as SLSA Provenance [154].

One of the earliest proposals to capture step operational metadata was in-toto [168] via a primitive referred to as Links. in-toto Links are *authenticated* documents describing the input and output artifacts to a step, the command executed during a step, and any step byproducts such as return value or error messages. Using principal and artifact defense mechanisms allows in-toto to also detect tampering with any aspect of the step (S3).

S3 - Interference with step detected: Detects attempts to tamper with or otherwise interfere with the expected behavior of a step. Detecting attempts to present different versions of the same step to different principals provide the stronger global non-equivocation property.

Step consensus relies on the repeatability and/or visibility of a step to enable multiple parties to determine jointly if a step has been tampered with (S3). That is, if different parties obtain different metadata or results for a given step, then the step is assumed to have been tampered with.

Approaches such as the IETF Supply Chain Integrity Transparency and Trust (SCITT [26]) framework and Ferraiuolo et al. [54] provide global (S2) and (S3) properties using mechanisms like distributed ledgers and transparency logs to expose policy decisions as part of the metadata recording and publication process.

Nikitin et al. proposed CHAINIAC [123], for instance, provides strong tamper-detection guarantees for software updates through a novel data structure called a skipchain (i.e., the merging of a skiplist and a blockchain). CHAINIAC detects step tampering in two stages. First, a group of build services re-runs a particular build step. Then, they validate the resulting binary, and only commit it into the skipchain if the builders reach consensus over the binary.

As CHAINIAC skipchains contain a cryptographically verifiable log of all released binaries, the approach also helps different parties ensure that they all have the same view of the software updates over time (the stronger non-equivocation property of S3).

4.5. Defending Supply Chain DAG Structure

The left-pad incident [146], in which the package's removal from the npm repository broke downstream build steps that use it as a dependency, exemplifies the

risks propagated via the relationships between supply chain elements. While previous sections cover individual supply chain elements, a small number of software supply chain defenses focuses on maintaining and securing their *transitive* relationships.

T1 - DAG changes detected. To validate the integrity of relationships between elements, changes to the structure of a supply chain DAG, i.e., additions, removals or reordering of individual elements, can be made detectable.

Supply chain layout integrity enables principals to detect whether all steps were performed in the expected order, whether they were carried out by the expected principals, and if artifacts were modified between steps. This approach first introduced in Torres-Arias et al. [168] reduces the risks of added, removed, or reordered steps.

Using a primitive called layouts, in-toto [168] implements this approach by requiring project owners or maintainers (a principal role) to pre-define the set of steps to be performed, the principals authorized to perform each step, and the expected consumed and produced materials for each step, a priori. Given an in-toto layout document authenticated by the expected project owner, and step metadata is available (see §4.4), interested principals can validate that an artifact was produced following the expected layout.

Beyond package dependency graphs, few mechanisms exist today to analyze the security and integrity of supply chain DAGs. More human-centered research is needed to understand how practitioners configure software supply chains DAGs, including all their elements, enabling a deeper understanding of how structural risks surface, and efforts to establish standard, robust DAG structures.

T2 - Consistent results. Checking whether a set of steps performed on artifacts in a specific order always produces consistent results can confirm the correctness of a DAG. Techniques that check for exact matches in final artifacts provide the strongest form of consistency.

Supply chain reproducibility, first proposed in the 1990s, seeks to detect whether a particular sequence of steps always produces the same results. In practice, reproducibility relies on the chaining of techniques for individual steps like deterministic builds (§4.4), which ensure that a particular step, such as build, always produces the same bit-for-bit result. By enabling downstream principals to repeat a sequence of upstream steps, upstream producers enable their consumers to also detect changes in the supply chain DAG for their product artifact.

As discussed in [57], most prior work has focused on addressing the technical challenges of reproducibility; more human-centered studies on the daily challenges of achieving and maintaining reproducibility are needed to improve adoption and lead to consistent supply chain DAGs.

Insight 3: We identified 13 security objectives for the core software supply chain elements (principals, artifacts, resources steps), and two extra objectives at the DAG structure level.

4.6. Prevalent Defense Patterns

Through our systematized security objectives and defense techniques, we observe prevalent strategies to how software supply chain defenses address risk *across* different elements. Thus, we derive six defense patterns, which we group into three general security control types [187].

More formally, **a defense pattern describes the scope, context and outcome of a defense technique**. That is, the security objective and target of a defense technique (its scope) commonly dictate when and where in the supply chain it must be implemented (its context). A defense technique's scope and context, in turn, affect its outcome, i.e., the time when a particular supply chain risk is identified and the extent to which it is mitigated.

We map our objectives in the taxonomy to the defense patterns in the last column group of table 2.

4.6.1. Detective Defenses

Approaches in this category are *reactive* as their outcome is that supply chain participants retroactively determine whether a threat occurred in the past. Such defense techniques are typically separated into two components: 1) a monitor or observer that collects security properties or behaviors of a supply chain element, and 2) a validator that is responsible for examining or verifying the collected supply chain information as at a later point in time. In the event a threat has occurred, the information collected by the monitor can serve as evidence to *detect* the problem and alert supply chain participants.

Transparency defenses have the goal of making any property about a supply chain element visible and verifiable by software supply chain principals.

All defenses that detect principal credential compromise (P2) offer transparency because they reveal confidentiality or integrity issues. Similarly, the observed artifact equivocation (A2) defenses, which provide artifact transparency, aim for supply chain elements to have the ability to be recognized when their views of an artifact diverge. Step transparency defenses record step operation information (S2), making software supply chain processes visible.

Integrity techniques seek to enable software supply chain principals to verify that a supply chain element is authentic, and compare two supply chain elements are equivalent.

Credential transparency not only provides enhanced auditability of principal credential usage, but also credential integrity by making mis-issued or tampered credentials detectable (P2). All techniques that detect changes in an artifact (A1) rely on cryptographic mechanisms to check artifact integrity, i.e., if it meets an expectation.

Similar to credential transparency, step transparency is a defense that covers both transparency and integrity goals by making interference with a supply chain step verifiably detectable (S3). Today's defenses, on the other hand, that aim to make software supply chain DAG changes detectable (T1) largely focus on providing integrity properties.

4.6.2. Corrective Defenses

These defenses are *active* as their outcome is to reduce the impact of actively occurring supply chain problems, including availability attacks. That is, even in the face of an attack or vulnerability, such defenses *correct* the supply chain element to a known good or stable state.

Like preventative defenses, corrective defenses are in place *a priori*, but they are triggered after an incident takes place.

Graceful degradation defenses have the objective to maintain a baseline level of security in the software supply chain, even when a compromise has occurred.

For principals, defenses that limit credential authority (**P3**) ensures that supply chain steps the a compromised credential is not authorized to access remain inaccessible. Similarly, mandatory access control in the software supply chain aims to maintain a baseline level of access restrictions to a resource (**R1**).

Redundancy techniques generally rely on a fallback mechanism to restore a particular supply chain element to normal function in the face of a compromise. The basis of all principal defenses requiring multiple principals to perform an authorization (**P4**) is to have sufficient uncompromised principals available in the event of an attack for a software supply chain step to be carried out as expected.

4.6.3. Preventative Defenses

These defenses are *proactive* as their outcome is to reduce the impact or prevent a specific threat from occurring altogether. Such techniques are applied before their target supply chain element is deployed in anticipation of potential risks, and thus they techniques follow a *preventative* pattern.

Limitation defenses restrict the privilege or side effects of a supply chain element with the goal of minimizing the surface for confidentiality or integrity attacks.

Ephemeral credentials constrain a compromised principal by limiting the use of credentials temporally (**P1**). Techniques that restrict the interface to an supply chain resource's environment (**R2**) or enforce strict environment boundaries (**R3**) spatially confine a resource so that it may not operate outside its expected scope.

Correctness defenses have the goal of ensuring correct *behavior* in supply chain steps by minimizing the occurrence of errors or exploitable weaknesses, and some even generating verifiable proofs of correctness.

All program analysis techniques to enforce expected artifact behavior (**A3**) examine an artifact's functionality to detect anomalous functionality, with some techniques like patch propagation and dynamic/ML-based analysis correcting vulnerable behaviors. Deterministic steps, on the other hand, enforce expected step behavior (**S1**) via specifying prior configurations that define correct and consistent step operations. Finally, correctness and consistent results across a supply chain DAG (**T2**) is achieved through reproducibility of an entire DAG, a technique that leverages defenses for all other supply chain elements.

Insight 4: Each software supply chain defense can be characterized by its **scope**, **context**, and **outcome**. We identify six recurring defense patterns, which we map into three high-level security control types: *detective*, *corrective*, and *preventative*.

5. Trends and Future Directions

Finally, we utilize the insights from above to derive information about the state of software supply chain defense practice to identify current trends and future work directions. These trends can shed light in existing gaps, as well as mis-allocated efforts which can in turn guide new research directions. In order to identify these gaps, we also utilize supply chain compromise data to ground our study in empiricism.

5.1. Attack Distribution Across AStRA Elements

To validate the expressiveness of AStRA model and the practical relevance of our security objectives, We analyze the attacks in the In-Q-Tel (IQT) lab's software supply chain compromises dataset[12], which is a foundational repository used across other software supply chain systematization literature such as Ladisa et al. [98]. Though other datasets exist (e.g., Ohm et al.), this dataset provides crucial information — the point of compromise — and various classess of software supply chain attacks (e.g., as opposed to malware samples). The dataset covers 183 incidents with detailed descriptions, categories, references, and timelines.

Building on this, we perform a comparative analysis between AStRA 's security objectives and the attack classes represented in the dataset. Given the natural with the Ladisa et al. SoK [98], we also utilize their software supply chain attack taxonomy to create a (*compromise*, *attack_vector*, *security_objective_violated*) triple. Grounding our evaluation in this dataset allows us to examine how documented compromises manifest across AStRA elements and the overall DAG structure, and to assess whether AStRA captures the breadth and nuance of adversarial behaviors observed in practice.

To perform this mapping, we identified the type of compromise each incident and matched it to the affected AStRA element as well as the violated security objective(s). For example, if the dataset reported key leakage or unauthorized account access, we labeled it as a principal compromise, if the cause involved a compromised CI/CD service, or build tool, we labeled it as a resource compromise, if it involves running malicious build, we labeled it as a step compromise. In cases where the compromise chain spanned multiple elements (e.g., a principal compromise leading to a resource compromise), we recorded all affected AStRA elements involved. For example, distributing a malicious version of a package to include remote access to users machines violates the expected behavior of an artifact (**A3**), and detecting artifacts changes (**A1**). Extra resources security objectives may have helped reducing this risk (e.g. limiting resource privileges and restricting environment interfaces «**R1**, **R2**»).

Our analysis demonstrates that all attacks in the dataset can be represented using the AStRA elements and mapped to the security objectives defined in our defense-oriented taxonomy. Aggregating the incidents by AStRA element reveals a pronounced skew in how supply chain compromises manifest. Artifact-level attacks overwhelmingly dominate, accounting for (82%), reflecting the prevalence of malicious package injection, and dependency hijacking. The second common category is resource related compromises (26%), which indicate adversarial pressure on CI/CD environments, privileged infrastructure, and isolation boundaries.

5.2. Security Objectives and Attack Trends

By encoding each attack as violations of security objectives, we observe the normalized distribution of attack trends across each security objective, as shown in figure 2. It is clear from the figure that most of the attacks concentrate on violating artifact and resources objectives, while fewer attacks target structural topology or step-level behaviors. The most frequently violated objective is A3 (Expected artifact behavior enforced), which appears in 81% of the incidents, making it by far the most heavily targeted property in real compromises. Respectively, Artifact Integrity is the second most targeted property (29%). Principal objectives (P1–P3) collectively account for 36% of the incidents, highlighting the persistent role of leaked, reused, or improperly scoped credentials. This alignment between attack patterns and our security objectives confirms that the objective set is expressive enough to capture known attack behavior and provides the foundation for systematically assessing whether available defenses can reduce the risk of these attacks to succeed.

Although academic defenses and industry tools differ in how heavily they invest in individual objectives, the overall shape of their distributions of defenses in Figure 2 mirrors the structure of the attack distribution. In all three views: attacks, academic work, and tooling: the dominant categories fall under artifacts, followed by resources, then principals, with considerably less activity in steps and topology. However, tooling shows notable priority divergences. For example, it overinvests in recording step operation (S2), allocating nearly 25% of its focus to an objective that appears in less than 1% of attacks. This overinvestment likely reflects the high perceived cost of pipeline failures, showing that defense priorities are influenced by the expected operational and economical impact. A possible explanation is that of the *impact* of a single incident on a particular property — this asymmetry warrants further research.

Insight 5: Academic defenses align nearly five times closer to real attack patterns than industrial tools, indicating that while research attention better reflects observed risk, more work is needed to bring tooling into alignment with that risk, particularly in enforcing expected artifact and step behavior (A3,S1) and in detecting interference with steps (S3).

Using this empirical data, we now discuss gaps in the software supply chain defense space, and discuss

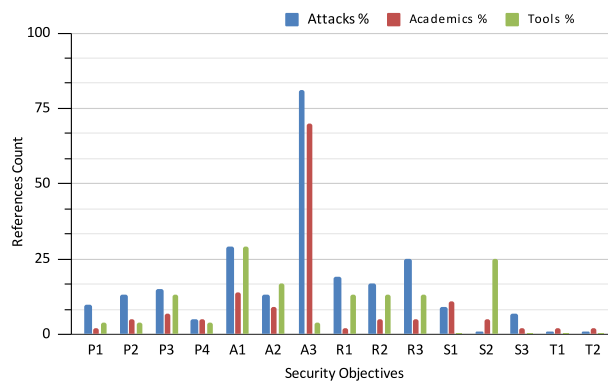


Figure 2. Normalized distribution of software supply chain attacks, academic, and tools references across security objectives

the high-level trend of security objectives addressed in the literature (from Fig. 2).

5.3. Principal discussion & Research Gaps

Table 2 reveals that no individual approach is sufficient to reduce the possible risks to the principals. Practical defenses have combined multiple approaches to provide stronger security properties. A promising combination of One Time Passwords (e.g., OTPs) with multifactor authentication (e.g., two-factor authentication [63]) reduces risks in two key areas (P1, P3). Despite the highly collaborative nature of today's software development practices, this widely adopted defense combination primarily reduces risks for individual principals, who represent single points of failure. Providing further security properties requires layering this approach with others to raise the bar to compromise an entire supply chain via a single principal.

Another promising combination of approaches are short-lived credentials like those provided by Sigstore's Fulcio [151] and credential transparency: in fact, Fulcio relies on CT [1] for the detection of compromise and the added reliability of short-lived credentials issued, reducing risks in three key areas (P1–P3). To raise usability, Fulcio relies on OpenID Connect [2] and timestamping services (RFC 3161) [15] for credential generation and discovery. Layering these defenses with additional approaches to address P4 would likely not impact adoption costs significantly.

Research gaps. Several approaches in our analysis have potential for addressing Principal risks, although they have been studied only in other application domains. Threshold schemes, for example, have been applied in other use cases such as private information retrieval (e.g., [177]) and peer-to-peer membership control (e.g., [118]). However, to our knowledge, their application in the software supply chain has yet to be explored. Threshold schemes, for instance, meet several principal security objectives (P3, P4); their applicability to improve the reliability of authorization in the supply chain is a promising future research direction.

In addition, credential transparency is a promising approach to detect credential compromise. However, its limited adoption for software supply chain security warrants study to determine its applicability in practice.

5.4. Artifacts discussion & Research Gaps

Perhaps the most noticeable aspect is the vast prevalence of **A3** as a security objective within software supply chain scholarship, as shown in Figure 2. Work in this domain is predominantly achieved by means of standard program analysis techniques. Refocusing efforts in other security objectives and techniques in the artifacts domain will likely be a fruitful pursuit.

With regard to techniques, Table 2 shows that there is no one-stop-shop to achieve all desired objectives, nor are several single approaches enough to achieve individual properties. Artifact and metadata signing, for example, are both needed to fully achieve **A1**. Static and dynamic analysis techniques are often combined for improved results to identify and enforce the expected artifact behavior (**A3**).

While composing systems to achieve all artifact security objectives may be feasible (e.g., by means of dynamic analysis and metadata signing), there is no established best practice for doing so, nor have the security implications of such combined systems been studied. This is a stark contrast with other elements of the model (see, e.g., the resources below), which have garnered significant industry interest in establishing guidance in the supply chain.

Research gaps. Some of the approaches evaluated have seen varying degrees of adoption. This is evidenced by the low adoption of solutions such as artifact and metadata signing in critical areas such as package management [6, 97, 148]. For hashing, for instance, research on mechanisms such as subresource integrity HTML tags [36] may address known usability issues [35], but more studies are needed on more defenses to understand adoption barriers.

Work establishing better mental models about trust in artifacts like packages through the above properties is lacking. In particular, security policies for software trust in different ecosystems are currently ad-hoc and do not follow a principled definition based on white or gray literature. That is, systems such as GPG have provided mechanisms for developers to sign packages, but they rarely develop infrastructure and processes to verify that the right actor signed for the right artifact. In addition, there is little research exploring the usability and process challenges on proper definition and enforcement of policies.

Approaches such as ledger-backed systems face performance and efficiency tradeoffs introduced by consensus algorithms used to help achieve **A2**. Given that these approaches can provide availability and privacy properties [113] in addition to meeting objectives (**A1**, **A2**), and these mechanisms provide strong deterrent to attackers, more research is needed to reduce their deployment cost.

5.5. Resources discussion & Research Gaps

As with prior supply chain elements, defense approaches for resources are complementary. However, unlike other areas of the chain, the long-standing history of computer systems security research has led to better compatibility between the evaluated approaches.

For instance, access control mechanisms are capable of operating at various levels of granularity depending on a resource's operating environment. Although implementations such as AppArmor [172] provide application-level MAC, DAC can provide system-level defenses. At the same time, filtering mechanisms, such as netfilter [186] can provide system-level protection in addition to DAC and MAC within the system. Virtualization then provides an additional level of isolation in addition to access control and filtering.

We observe that resource defense approaches appear more frequently in industry security best-practice documents that have been authored in response to recent supply chain compromises. Most notably, the Cloud Native Compute Foundation's Software Supply Chain Best Practices [38] and the Secure Software Development Framework (SSDF) by NIST [156] both describe a set of recommendations for configuring and implementing more secure software supply chain resources that rely on defense approaches including MAC, filtering, and virtualization. The Supply Chain Levels for Software Artifacts (SLSA) [154], which provides a framework for incrementally implementing a hardened build pipeline, also includes requirements for resource isolation to ensure that build steps are performed without external influence to meet higher security levels.

Research gaps. Academic research analyzing the security of software supply chain resources thus far taken primarily an attack-oriented view (e.g., [49]). Although the design and application of access control and resource isolation techniques have been widely studied in the academic systems community, research dedicated to applying these defense mechanisms in the context of software supply chain security is lacking; we see ample opportunities to build upon defense-oriented academic studies such as Gruhn et al. [69] and Ohm et al. [131].

For brevity, we do not extensively evaluate resource defense approaches such as bounds checking (e.g., [176]), or more recent hardware-based techniques such as trusted execution environments (e.g., [16, 21, 82, 111]). However, we observe in existing best practices guides that such approaches are underexplored in supply chain settings. In particular, the inclusion of hardware-based attestation primitives along with software/provenance attestation seems like a natural construction to explore.

The focus on reducing risks of building and packaging resources [69, 131, 154] leaves an opportunity for future research to explore existing and novel approaches for other software supply chain use cases. More research is needed on the adoption of industry best practices.

5.6. Steps Discussion & Research Gaps

The majority of step defenses work has focused on mechanisms for recording step metadata to detect tampering. Xia et al. [190], for instance, conducted a study that focuses on current SBOM practices, tooling and concerns from SBOM practitioners' perspectives. The study shows lack of SBOM adoption and problems in enabling SBOMs benefits, due to various concerns

regarding SBOM generation, distribution, consumption processes.

Research gaps. In both academic and gray literature, we observe a general lack of studies on preventing or reducing risks that affect the correct execution or operation of software supply chain steps. Formal verification, for instance, has been widely explored by the programming language and software engineering communities, though its application in the context of the software supply chain remains underexplored. Furthermore, the adoption of formal verification in resources for steps is a promising research opportunity.

Although the design of consensus-based step defense mechanisms has been the focus of an increasing number of academic studies, their practical applicability and adoption costs remain relatively underexplored. Similarly, the design, development, and adoption of step containment approaches at large is an area that warrants further study.

5.7. Other Salient Points & Research Gaps

The software supply chain DAG shares many characteristics with multi-party, distributed systems. Given the focus on securing individual supply chain elements, software supply chain security would greatly benefit from more applying knowledge and from distributed systems, such as fault tolerance and replication, to address transitive risks.

Reducing Barriers to Adoption. Originally, we intended to evaluate the tradeoffs and costs to adopt mechanisms for software supply chain security. Unfortunately, we found there are very little literature and formal studies on how to deploy a particular approach. These studies should cover human factors (e.g., training), usability, and system requirements (e.g., extra servers). Thus, we envision a green field for research in *Usable software supply chain security*.

Moving beyond modeling threats. Our objective-driven model is a first step towards systematically mapping high-level security objectives and patterns to defenses. We see an opportunity to expand upon our work to design novel adoption-ready software supply chain defenses. Beyond our preliminary work, we envision future research that incorporates holistic cost-benefit analysis frameworks for supply chain risk mitigation with the goal of increasing adoption of supply chain defenses.

Understanding the impact & security of AI-generated software. Generative AI is playing an increasingly central role in software development. According to a recent survey [149], about 92% of developers in the U.S. rely on AI-based tools. This rapid shift presents ample future research opportunities to study the security implications of AI-generated code and address software and supply chain security in this space.

Insight 6: There are four significant opportunities: 1) Studying defense adoption and usability. 2) Developing deployable defenses grounded in human factors. 3) Exploring foundational tools (e.g., trusted execution environments) in supply chain contexts. 4) Formalizing trust models and policies for artifact provenance.

6. Conclusions

This paper reframes the security of the software supply chain from an attacker's checklist to a defender's blueprint. By modeling chains as DAGs of principals, artifacts, resources, and steps, we distill fifteen cross-cutting security objectives and map academic and industrial mechanisms onto them, exposing three recurring defense patterns. We further explore how these patterns surface possible secure configurations, and note trends in existing literature in the area of software supply chain. With this in mind, we also surface promising but underexplored lines of work.

By making shortcomings explicit and placing existing defenses on a common scaffold, we provide researchers with a roadmap of open problems, practitioners with guidance for defense prioritization, and policymakers with levers for measurable requirements. We hope that this objective-driven perspective catalyzes a new generation of holistic, composable defenses capable of keeping pace with the dynamic, federated nature of today's software supply chains.

Ethics Considerations and Open Science

To support open science principles and ensure transparency, artifacts produced in this study, including datasets and data collection scripts, are released under an open source license and made publicly accessible. This enables verification and reproducibility of the results. Ethical considerations were taken into account throughout the work: no proprietary or sensitive data were used, and all third-party components were used according to their respective license terms.

Acknowledgments

This material is based on work supported in part by the Air Force Office of Scientific Research (AFOSR) under award number FA9550-24-1-0015; by the National Center for Transportation Cybersecurity and Resiliency (TraCR) under grant numbers 69A3552344812 and 69A3552348317; and by a grant from the Digital Infrastructure Insights Fund, a fiscally sponsored project of Aspiration.

References

- [1] Certificate Transparency Version. <https://certificate.transparency.dev/>, retrieved Apr. 2023.
- [2] How OpenID Connect Works. <https://openid.net/connect/>.
- [3] in-toto Attestation Framework. <https://github.com/in-toto/attestation>. original-date: 2021-04-01T14:58:27Z.
- [4] left-pad/left-pad. <https://github.com/left-pad/left-pad>. original-date: 2014-03-14T09:09:34Z.

- [5] [meta] Binary Transparency on Firefox. https://bugzilla.mozilla.org/show_bug.cgi?id=1341395.
- [6] PGP signatures on PyPI: worse than useless. <https://blog.yossarian.net/2023/05/21/PGP-signatures-on-PyPI-worse-than-useless>.
- [7] ReproducibleBuilds/BuildinfoFiles - Debian Wiki. <https://wiki.debian.org/ReproducibleBuilds/BuildinfoFiles>.
- [8] SELinux Project. <https://github.com/selinuxproject>.
- [9] SolarWinds hack was 'largest and most sophisticated attack' ever: Microsoft president.
- [10] Apple's App Store infected with XcodeGhost malware in China. BBC News, Sept. 2015.
- [11] GitHub · Build and ship software on a single, collaborative platform. <https://github.com/>, 2025.
- [12] IQTLabs/software-supply-chain-compromises, Aug. 2025. original-date: 2020-09-10T12:47:33Z.
- [13] Advanced Micro Devices Inc. AMD64 Architecture Programmer's Manual Volume 2: System Programming. https://www.cs.wm.edu/~smherwig/readings/manuals/amd/sdm/amd64_arch_programmers_manual-vol2-system_programming.pdf, retrieved Dec. 2023, Jun 2023.
- [14] M. Abbadini, D. Facchinetti, G. Oldani, M. Rossi, and S. Paraboschi. NatiSand: Native code sandboxing for JavaScript runtimes. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, RAID '23*, pages 639–653. ACM, Oct. 2023.
- [15] C. Adams, P. Cain, D. Pinkas, and R. Zuccherato. Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). = <https://www.ietf.org/rfc/rfc3161.txt>.
- [16] Advanced Micro Devices, Inc. AMD Secure Encrypted Virtualization (SEV). <https://developer.amd.com/sev/>.
- [17] H. Afzali, S. Torres-Arias, R. Curtmola, and J. Cappos. Legitimate: Towards verifiable web-based git repositories. In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security, ASIACCS '18*, page 469–482, New York, NY, USA, 2018. ACM.
- [18] M. Al-Bassam and S. Meiklejohn. Contour: A Practical System for Binary Transparency.
- [19] N. Albartus, C. Nasenberger, F. Stolz, M. Fyrbiak, C. Paar, and R. Tessier. On the Design and Misuse of Microcoded (Embedded) Processors — A Cautionary Note. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 267–284. USENIX Association, Aug. 2021.
- [20] A. AlHamdan and C.-A. Staicu. SandDriller: A Fully-Automated approach for testing Language-Based JavaScript sandboxes. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3457–3474, Anaheim, CA, Aug. 2023. USENIX Association.
- [21] T. Alves. Trustzone : Integrated hardware and software security. 2004.
- [22] A. S. Ami, N. Cooper, K. Kafle, K. Moran, D. Poshyvanyk, and A. Nadkarni. Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 614–631, 2022.
- [23] Apache Software Foundation. CVE-2021-44832. <https://nvd.nist.gov/vuln/detail/CVE-2021-44832>, retrieved Jul. 2022.
- [24] Aqua Security Software Ltd. Aqua Trivy: Vulnerability and Misconfiguration Scanning. <https://www.aquasec.com/products/trivy/>.
- [25] A. Bandyopadhyay. The Software Release Life Cycle in DevOps: Why Reinvent The Model? <https://linuxhandbook.com/software-release-life-cycle-devops/>.
- [26] H. Birkholz, A. Delignat-Lavaud, C. Fournet, and Y. Deshpande. An Architecture for Trustworthy and Transparent Digital Supply Chains. <https://datatracker.ietf.org/doc/draft-birkholz-scitt-architecture/>, Oct 2022.
- [27] A. Birsan. Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies. <https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610>, Sept. 2023.
- [28] N. Boucher and R. Anderson. Trojan source: invisible vulnerabilities. In *Proceedings of the 32nd USENIX Conference on Security Symposium, SEC '23*, USA, 2023. USENIX Association.
- [29] L. Boughton, C. Miller, Y. Acar, D. Wermke, and C. Kästner. Decomposing and measuring trust in open-source software supply chains. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, ICSE-NIER'24*, page 57–61, New York, NY, USA, 2024. ACM.
- [30] A. Bytes, P. H. N. Rajput, C. Doumanidis, M. Maniatakos, J. Zhou, and N. O. Tippenhauer. FieldFuzz: In Situ Blackbox Fuzzing of Proprietary Industrial Automation Runtimes via the Network. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, pages 499–512, Hong Kong China, Oct. 2023. ACM.
- [31] M. Campobasso and L. Allodi. Impersonation-as-a-service: Characterizing the emerging criminal infrastructure for user impersonation at scale. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS '20*, page 1665–1680, New York, NY, USA, 2020. ACM.
- [32] J. Cappos, J. Samuel, S. Baker, and J. H. Hartman. A look in the mirror: attacks on package managers. In *Proceedings of the 15th ACM conference on Computer and communications security - CCS '08*, page 565. ACM Press.
- [33] R. Chandramouli, F. Kautz, and S. Torres-Arias. Strategies for the Integration of Software Supply Chain Security in DevSecOps CI/CD Pipelines. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204D.pdf>, 2024.
- [34] Y. Chen, Q. Wang, Y. Yang, Y. Chen, Y. Li, and S. Ji. Unveiling Security Vulnerabilities in Git Large File Storage Protocol . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 468–485, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
- [35] M. Cherubini, A. Meylan, B. Chapuis, M. Humbert, I. Bilogrevic, and K. Huguenin. Towards usable checksums: Automating the integrity verification of web downloads for the masses. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1256–1271. ACM.
- [36] M. Cherubini, A. Meylan, B. Chapuis, M. Humbert, I. Bilogrevic, and K. Huguenin. Towards usable checksums: Automating the integrity verification of web downloads for the masses. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1256–1271, 2018.
- [37] J. Christensen, I. M. Anghel, R. Taglang, M. Chiroiu, and R. Sion. DECAF: Automatic, adaptive de-bloating and hardening of COTS firmware. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 1713–1730. USENIX Association, Aug. 2020.
- [38] CNCF Security TAG. Software Supply Chain Best Practices v2. <https://github.com/cncf/tag-security/blob/main/community/working-groups/supply-chain-security/supply-chain-security-paper-v2/SSCBPv2.md>, retrieved June 2025.
- [39] E. Commission. Cyber resilience act. <https://digital-strategy.ec.europa.eu/en/library/cyber-resilience-act>, 2022.
- [40] E. Cornelissen, M. Shcherbakov, and M. Balliu. GHunter: Universal prototype pollution gadgets in JavaScript runtimes. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3693–3710, Philadelphia, PA, Aug. 2024. USENIX Association.
- [41] L. Courtès. Functional package management with guix. *arXiv preprint arXiv:1305.4584*, 2013.
- [42] V. Craciun, P. A. Felber, A. Mogage, E. Onica, and R. Pires. Malware in the SGX supply chain: Be careful when signing enclaves! *IEEE Transactions on Dependable and Secure Computing*, pages 1–1, 2020.
- [43] Debian. SecureApt. <https://wiki.debian.org/SecureApt>, retrieved Dec. 2023.
- [44] H. O. Delicheh, A. Decan, and T. Mens. Quantifying security issues in reusable javascript actions in github workflows. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, pages 692–703, 2024.
- [45] H. Denbraver. Malicious packages found to be typo-squatting in Python Package Index. <https://snyk.io/blog/malicious-packages-found-to-be-typo-squatting-in-pypi/>, Dec 2019.
- [46] E. Derr, S. Bugiel, S. Fahl, Y. Acar, and M. Backes. Keep me updated: An empirical study of third-party library updatability on android. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS '17*, page 2187–2200, New York, NY, USA, 2017. ACM.
- [47] Docker inc. Docker Content Trust . <https://docs.docker.com/engine/security/trust/>.
- [48] F. Draschbacher and L. Maar. Manifest problems: Analyzing code transparency for android application bundles. In *2024 Annual Computer Security Applications Conference (ACSAC)*, pages 1109–1122, 2024.
- [49] R. Duan, O. Alrawi, R. P. Kasturi, R. Elder, B. Saltaformaggio, and

- W. Lee. Towards measuring supply chain attacks on package managers for interpreted languages. In Proceedings of the 28th Annual Network and Distributed System Security Symposium (NDSS), 2021.
- [50] T. Dunlap, E. Lin, W. Enck, and B. Reaves. VFCFinder: Pairing Security Advisories and Patches. In Proceedings of the 19th ACM Asia Conference on Computer and Communications Security, pages 1128–1142, Singapore Singapore, July 2024. ACM.
- [51] S. Dupont, A. Yautsiukhin, G. Ginis, G. Iadarola, S. Fagnano, F. Martinelli, C. Ponsard, A. Legay, and P. Massonet. Product Incremental Security Risk Assessment Using DevSecOps Practices. In S. Katsikas, F. Cuppens, C. Kalloniatis, J. Mylopoulos, F. Pallas, J. Pohle, M. A. Sasse, H. Abie, S. Ranise, L. Verderame, E. Cambiaso, J. Maestre Vidal, M. A. Sotelo Monge, M. Albanese, B. Katt, S. Pirbhulal, and A. Shukla, editors, Computer Security, ESORICS 2022 International Workshops, pages 666–685, Cham, 2023. Springer International Publishing.
- [52] eBPFio Authors. What is eBPF? <https://ebpf.io/what-is-ebpf/>, retrieved Dec. 2023, 2023.
- [53] C. C. Editor. Isolation - glossary | CSRC. <https://csrc.nist.gov/glossary/term/isolation>.
- [54] A. Ferraiuolo, R. Behjati, T. Santoro, and B. Laurie. Policy transparency: Authorization logic meets general transparency to prove software supply chain integrity. In Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses, pages 3–13. ACM.
- [55] FireEye. Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor. FireEye Blogs - Threat Research, Dec 2020.
- [56] A. Foundation. Working with gpg. <https://central.sonatype.org/publish/requirements/gpg/>.
- [57] M. Fourné, D. Wermke, W. Enck, S. Fahl, and Y. Acar. It's like flossing your teeth: On the importance and challenges of reproducible builds for software supply chain security. In 44th IEEE Symposium on Security and Privacy, 2023.
- [58] J. Gamba, M. Rashed, A. Razaghpanah, J. Tapiador, and N. Vallina-Rodriguez. An analysis of pre-installed android software. In 2020 IEEE Symposium on Security and Privacy (SP), pages 1039–1055, 2020.
- [59] Git. Git Basics - Viewing the Commit History. <https://git-scm.com/book/en/v2/Git-Basics-Viewing-the-Commit-History>, retrieved Dec. 2023.
- [60] Git SCM. Signing your work. <https://git-scm.com/book/en/v2/Git-Tools-Signing-Your-Work>.
- [61] GitHub Inc. About commit signature verification. <https://docs.github.com/en/authentication/managing-commit-signature-verification/about-commit-signature-verification>.
- [62] GitHub, Inc. Repository roles for an organization, Oct 2022.
- [63] GitHub Inc. Accessing GitHub using two-factor authentication. <https://docs.github.com/en/authentication/securing-your-account-with-two-factor-authentication-2fa/accessing-github-using-two-factor-authentication>, retrieved June 2025, 2025.
- [64] M. F. Gobbi and J. Kinder. Poster: Using CodeQL to Detect Malware in npm. In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pages 3519–3521, Copenhagen Denmark, Nov. 2023. ACM.
- [65] D. Gonzalez, T. Zimmermann, P. Godefroid, and M. Schaefer. Anomalous: Automated detection of anomalous and potentially malicious commits on github. In 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pages 258–267, 2021.
- [66] D. Goodin. Hackers backdoor PHP source code after breaching internal git server. <https://arstechnica.com/gadgets/2021/03/hackers-backdoor-php-source-code-after-breaching-internal-git-server/>, Mar 2021.
- [67] D. Goodin. Go module mirror served backdoor to devs for 3+ years. <https://arstechnica.com/security/2025/02/backdoored-package-in-go-mirror-site-went-unnoticed-for-3-years/>, February 2025. Accessed: 2025-11-21.
- [68] A. Goyal, G. Wang, and A. Bates. R-caid: Embedding root cause analysis within provenance-based intrusion detection. In 2024 IEEE Symposium on Security and Privacy (SP), pages 3515–3532, 2024.
- [69] V. Gruhn, C. Hannebauer, and C. John. Security of public continuous integration services. In Proceedings of the 9th International Symposium on Open Collaboration, WikiSym '13, New York, NY, USA, 2013. ACM.
- [70] Y. Gu, L. Ying, H. Chai, Y. Pu, H. Duan, and X. Gao. More Haste, Less Speed: Cache Related Security Threats in Continuous Integration Services. In 2024 IEEE Symposium on Security and Privacy (SP), pages 1179–1197, Los Alamitos, CA, USA, May 2024. IEEE Computer Society.
- [71] Y. Gu, L. Ying, H. Chai, C. Qiao, H. Duan, and X. Gao. Continuous intrusion: Characterizing the security of continuous integration services. In 2023 IEEE Symposium on Security and Privacy (SP), pages 1561–1577, 2023.
- [72] Y. Gu, L. Ying, Y. Pu, X. Hu, H. Chai, R. Wang, X. Gao, and H. Duan. Investigating package related security threats in software registries. In 2023 IEEE Symposium on Security and Privacy (SP), pages 1578–1595, 2023.
- [73] X. Han, X. Yu, T. Pasquier, D. Li, J. J. Rhee, J. W. Mickens, M. I. Seltzer, and H. Chen. Sigl: Securing software installations through deep graph learning. In USENIX Security Symposium, 2020.
- [74] M. S. Haq, T. D. Nguyen, A. Tosun, F. Vollmer, T. Korkmaz, and A.-R. Sadeghi. Sok: A comprehensive analysis and evaluation of docker container attack and defense mechanisms. In 2024 IEEE Symposium on Security and Privacy (SP), pages 4573–4590, 2024.
- [75] Y. He, Z. Zou, K. Sun, Z. Liu, K. Xu, Q. Wang, C. Shen, Z. Wang, and Q. Li. RapidPatch: Firmware hotpatching for Real-Time embedded devices. In 31st USENIX Security Symposium (USENIX Security 22), pages 2225–2242, Boston, MA, Aug. 2022. USENIX Association.
- [76] K. Herzig, L. Ghostling, M. Grothusmann, S. Just, N. Huang, A. Klimowski, Y. Ramkumar, M. McLeroy, K. Muslu, H. Sajani, and V. Vadaga. Microsoft cloudmine: Data mining for the executive order on improving the nation's cybersecurity. In 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR), pages 639–639, 2022.
- [77] C. Huang, N. Wang, Z. Wang, S. Sun, L. Li, J. Chen, Q. Zhao, J. Han, Z. Yang, and L. Shi. DONAPI: Malicious NPM packages detector using behavior sequence knowledge mapping. In 33rd USENIX Security Symposium (USENIX Security 24), pages 341–358, Philadelphia, PA, Aug. 2024. USENIX Association.
- [78] K. Huang, C. Lu, Y. Cao, B. Chen, and X. Peng. VMud: Detecting Recurring Vulnerabilities with Multiple Fixing Functions via Function Selection and Semantic Equivalent Statement Matching. In Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, pages 3958–3972, Salt Lake City UT USA, Dec. 2024. ACM.
- [79] A. Ilgayev. How We Discovered Vulnerabilities in CI/CD Pipelines of Popular Open-Source Projects. <https://cycode.com/blog/github-actions-vulnerabilities/>, Mar. 2022.
- [80] in-toto Authors. Create in-toto layout. <https://in-toto.engineering.nyu.edu/>, retrieved Apr. 2023.
- [81] in-toto Authors. in-toto - A framework to secure the integrity of software supply chains. <https://in-toto.io/>.
- [82] Intel Corporation. Intel® Trust Domain Extensions (Intel® TDX). <https://software.intel.com/content/www/us/en/develop/articles/intel-trust-domain-extensions.html>.
- [83] International Organization for Standardization, International Electrotechnical Commission. ISO/IEC 27034-1:2011 Information technology — Security techniques — Application security — Part 1: Overview and concepts, Nov 2011.
- [84] A. Jia, M. Fan, X. Xu, W. Jin, H. Wang, and T. Liu. Cross-inlining binary function similarity detection. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, New York, NY, USA, 2024. ACM.
- [85] L. Jiang, J. An, H. Huang, Q. Tang, S. Nie, S. Wu, and Y. Zhang. Binaryai: Binary software composition analysis via intelligent binary source code matching. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, New York, NY, USA, 2024. ACM.
- [86] K. R. Jones, T.-F. Yen, S. C. Sundaramurthy, and A. G. Bardas. Deploying android security updates: an extensive study involving manufacturers, carriers, and end users. In Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS '20, page 551–567, New York, NY, USA, 2020. ACM.
- [87] G. Jungwirth, A. Saha, M. Schröder, T. Fiebig, M. Lindorfer, and J. Cito. Connecting the .dotfiles: Checked-in secret exposure with extra (lateral movement) steps. In 2023 IEEE/ACM

- 20th International Conference on Mining Software Repositories (MSR), pages 322–333, 2023.
- [88] T. Karthik, D. McCoy, S. Lauzon, A. Brown, S. Awwad, R. Bielawski, C. Mott, A. Weimerskirch, and J. Cappos. Uptane: Securing Software Updates for Automobiles.
- [89] R. P. Kasturi, J. Fuller, Y. Sun, O. Chabklo, A. Rodriguez, J. Park, and B. Saltaformaggio. Mistrust plugins you must: A Large-Scale study of malicious plugins in WordPress marketplaces. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 161–178, Boston, MA, Aug. 2022. USENIX Association.
- [90] S. Klivan, S. Höltervenhoff, R. Pankus, K. Marky, and S. Fahl. Everyone for themselves? a qualitative study about individual security setups of open source software contributors. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1065–1082, 2024.
- [91] B. Kloeg, A. Y. Ding, S. Pellegrini, and Y. Zhauniarovich. Charting the path to sbom adoption: A business stakeholder-centric approach. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security, ASIA CCS '24*, page 1770–1783, New York, NY, USA, 2024. Association for Computing Machinery.
- [92] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry. Characterizing the security of github {CI} workflows. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2747–2763, 2022.
- [93] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry. Characterizing the Security of Github CI Workflows. In *Proc. USENIX Security '22*, Aug 2022.
- [94] A. Krause, J. H. Klemmer, N. Huaman, D. Wermke, Y. Acar, and S. Fahl. Pushed by accident: A Mixed-Methods study on strategies of handling secret information in source code repositories. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 2527–2544, Anaheim, CA, Aug. 2023. USENIX Association.
- [95] D. Krohmer, K. Sharma, and S. Chen. Adapting Static Taint Analyzers to Software Marketplaces: A Leverage Point for Mass Vulnerability Detection? In *Proc. ACM SCORED '22*, Nov 2022.
- [96] T. K. Kuppusamy, V. Diaz, J. Cappos, and M. Moore. Pep 480 – surviving a compromise of pypi: End-to-end signing of packages. <https://peps.python.org/pep-0480/>.
- [97] T. K. Kuppusamy, S. Torres-Arias, V. Diaz, and J. Cappos. Diplomat: Using delegations to protect community repositories. In *13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, pages 567–581, 2016.
- [98] P. Ladisa, H. Plate, M. Martinez, and O. Barais. Sok: Taxonomy of attacks on open-source software supply chains. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1509–1526, 2023.
- [99] P. Ladisa, H. Plate, M. Martinez, O. Barais, and S. E. Ponta. Towards the Detection of Malicious Java Packages. In *Proc. ACM SCORED '22*, Nov 2022.
- [100] P. Ladisa, S. E. Ponta, N. Ronzoni, M. Martinez, and O. Barais. On the Feasibility of Cross-Language Detection of Malicious Packages in npm and PyPI. In *Annual Computer Security Applications Conference*, pages 71–82, Austin TX USA, Dec. 2023. ACM.
- [101] Y. Landeck, D. Balta, M. Wimmer, and C. Knierim. Software in the manufacturing industry: Emerging security challenge areas for iiot platforms. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP '24*, page 118–121, New York, NY, USA, 2024. ACM.
- [102] X. Li, Y. Gu, C. Qiao, Z. Zhang, D. Liu, L. Ying, H. Duan, and X. Gao. Toward understanding the security of plugins in continuous integration services. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24*, page 482–496, New York, NY, USA, 2024. ACM.
- [103] S. Liao, L. Cheng, X. Luo, Z. Song, H. Cai, D. D. Yao, and H. Hu. A first look at security and privacy risks in the rapidapi ecosystem. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24*, page 1626–1640, New York, NY, USA, 2024. ACM.
- [104] R. Lorch, D. Larraz, C. Tinelli, and O. Chowdhury. A comprehensive, automated security analysis of the uptane automotive over-the-air update framework. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses, RAID '24*, page 594–612, New York, NY, USA, 2024. ACM.
- [105] A. Lounis, A. Andreoli, M. Debbabi, and A. Hanna. Seum spread: Discerning security flaws in iot firmware via call sequence semantics. In *Detection of Intrusions and Malware, and Vulnerability Assessment: 21st International Conference, DIMVA 2024, Lausanne, Switzerland, July 17–19, 2024, Proceedings*, page 395–415, Berlin, Heidelberg, 2024. Springer-Verlag.
- [106] C. Luo, W. Meng, and S. Wang. Strengthening supply chain security with fine-grained safe patch identification. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. ACM.
- [107] A. Machiry, N. Redini, E. Camellini, C. Kruegel, and G. Vigna. Spider: Enabling fast patch propagation in related software repositories. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 1562–1579, 2020.
- [108] J. Malka. Increasing trust in the open source supply chain with reproducible builds and functional package management. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE-Companion '24*, page 184–186, New York, NY, USA, 2024. ACM.
- [109] J. Malka, S. Zacchiroli, and T. Zimmermann. Reproducibility of build environments through space and time. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, ICSE-NIER'24*, page 97–101, New York, NY, USA, 2024. ACM.
- [110] A. F. Marshall. Announcing the Secure Software Factory Reference Architecture Paper. Cloud Native Compute Foundation Blog, May 2022.
- [111] F. McKeen, I. Alexandrovich, A. Berenzon, C. V. Rozas, H. Shafi, V. Shanbhogue, and U. R. Savagaonkar. Innovative Instructions and Software Model for Isolated Execution. In *Proc. Hardware and Architectural Support for Security and Privacy*, 2013.
- [112] X. Meng, B. P. Miller, and K.-S. Jun. Identifying Multiple Authors in a Binary Program. In S. N. Foley, D. Gollmann, and E. Sneekenes, editors, *Computer Security – ESORICS 2017*, pages 286–304, Cham, 2017. Springer International Publishing.
- [113] K. Merrill, Z. Newman, S. Torres-Arias, and K. R. Sollins. Speranza: Usable, Privacy-friendly Software Signing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS '23*, pages 3388–3402, New York, NY, USA, 2023. ACM.
- [114] E. Milner and R. Kosef. CIS Software Supply Chain Security Guide. <https://learn.cisecurity.org/Software-Supply-Chain-Security-Guide-v1-0>, 2022.
- [115] M. Moore, T. K. Kuppusamy, and J. Cappos. Artemis: Defanging software supply chain attacks in multi-repository update systems. In *Proceedings of the 39th Annual Computer Security Applications Conference, ACSAC '23*, page 83–97, New York, NY, USA, 2023. ACM.
- [116] M. Moore, A. S. A. Yelgundhalli, T. K. Kuppusamy, S. Torres-Arias, L. A. DeLong, and J. Cappos. Scudo: A proposal for resolving software supply chain insecurities in vehicles. 2022.
- [117] S. Muralee, I. Koishybayev, A. Nahapetyan, G. Tystahl, B. Reaves, A. Bianchi, W. Enck, A. Kapravelos, and A. Machiry. ARGUS: A Framework for Staged Static Taint Analysis of GitHub Workflows and Actions. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6983–7000, 2023.
- [118] M. Narasimha, G. Tsudik, and J. H. Yi. On the utility of distributed cryptography in p2p and manets: the case of membership control. In *11th IEEE International Conference on Network Protocols, 2003. Proceedings.*, pages 336–345, 2003.
- [119] O. S. Navarro Leija, K. Shiptoski, R. G. Scott, B. Wang, N. Renner, R. R. Newton, and J. Devietti. Reproducible containers. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 167–182. ACM.
- [120] S. Neupane, G. Holmes, E. Wyss, D. Davidson, and L. De Carli. Beyond typosquatting: an in-depth look at package confusion. In *Proceedings of the 32nd USENIX Conference on Security Symposium, SEC '23*, USA, 2023. USENIX Association.
- [121] Z. Newman, J. S. Meyers, and S. Torres-Arias. Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22*, page 2353–2367, New York, NY, USA, 2022. ACM.
- [122] D. C. Nguyen, E. Derr, M. Backes, and S. Bugiel. Up2Dep:

- Android Tool Support to Fix Insecure Code Dependencies. In *Annual Computer Security Applications Conference*, pages 263–276, Austin USA, Dec. 2020. ACM.
- [123] P. Nikitin, P. Jovanovic, B. Ford, et al. CHAINIAC: Proactive software-update transparency via collectively signed skipchains and verified builds. In *Proceedings of the 26th USENIX Security Symposium (USENIX Security '17)*, pages 127–144. USENIX Association, 2017. Accessed: 6 Jun 2025.
- [124] N. Nino, R. Lu, W. Zhou, K. H. Lee, Z. Zhao, and L. Guan. Unveiling IoT security in reality: A Firmware-Centric journey. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5609–5626, Philadelphia, PA, Aug. 2024. USENIX Association.
- [125] npm Contributors. Configuring two-factor authentication. <https://docs.npmjs.com/configuring-two-factor-authentication>, retrieved June 2025, 2025.
- [126] npm, Inc. Organization roles and permissions. <https://docs.npmjs.com/organization-roles-and-permissions>, retrieved Apr. 2023.
- [127] npm Registry. left-pad (npm package). <https://www.npmjs.com/package/left-pad>, 2018. Version 1.3.0; accessed 6 Jun 2025.
- [128] G. Ntousakis, S. Ioannidis, and N. Vasilakis. Demo: Detecting Third-Party Library Problems with Combined Program Analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2429–2431, Virtual Event Republic of Korea, Nov. 2021. ACM.
- [129] M. Ogata, J. Franklin, J. Voas, V. Sritapan, and S. Quirolgico. Vetting the Security of Mobile Applications. National Institute of Standards and Technology, Gaithersburg, MD), NIST Special Publication (SP) 800-164, Rev 1, April 2019.
- [130] M. Ohm, H. Plate, A. Sykosch, and M. Meier. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment*, 2020.
- [131] M. Ohm and C. Stuke. Sok: Practical detection of software supply chain attacks. In *Proceedings of the 18th International Conference on Availability, Reliability and Security, ARES '23*, New York, NY, USA, 2023. ACM.
- [132] C. Okafor, T. R. Schorlemmer, S. Torres-Arias, and J. C. Davis. Sok: Analysis of software supply chain security by establishing secure design properties. In *Proceedings of the 2022 ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses, SCORED'22*, page 15–24, New York, NY, USA, 2022. ACM.
- [133] OpenSSF Best Practices Working Group. OpenSSF Scorecard. <https://securityscorecards.dev/>.
- [134] OSCAL NIST team. Open Security Controls Assessment Language (OSCAL). <https://pages.nist.gov/OSCAL/>.
- [135] Z. Pan, X. Hu, X. Xia, X. Zhan, D. Lo, and X. Yang. Ppt4j: Patch presence test for java binaries. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. ACM.
- [136] N. Pantelaios and A. Kapravelos. FV8: A forced execution JavaScript engine for detecting evasive techniques. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 3747–3764, Philadelphia, PA, Aug. 2024. USENIX Association.
- [137] D. Pinckney, E. Cassano, A. Guha, and J. Bell. A Large Scale Analysis of Semantic Versioning in NPM. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 485–497, Los Alamitos, CA, USA, May 2023. IEEE Computer Society.
- [138] S. E. Ponta, H. Plate, A. Sabetta, M. Bezzi, and C. Dangremont. A manually-curated dataset of fixes to vulnerabilities of open-source software. In *Proceedings of the 16th International Conference on Mining Software Repositories, MSR '19*, page 383–387, Montreal, Quebec, Canada, 2019. IEEE Press.
- [139] A. Possemato, S. Aonzo, D. Balzarotti, and Y. Fratantonio. Trust, but verify: A longitudinal analysis of android oem compliance and customization. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 87–102, 2021.
- [140] J. Rack and C.-A. Staicu. Jack-in-the-box: An empirical study of javascript bundling on the web and its security implications. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS '23*, page 3198–3212, New York, NY, USA, 2023. ACM.
- [141] S. Ramakrishna. An investigative update of the cyberattack. <https://orangematter.solarwinds.com/2021/05/07/an-investigative-update-of-the-cyberattack/>.
- [142] Red Hat. Urgent security alert for fedora linux 40 and fedora rawhide users. <https://www.redhat.com/en/blog/urgent-security-alert-fedora-41-and-rawhide-users>, 2024. Accessed: 6 Jun 2025.
- [143] A. Roberts, M. Malayjerdi, M. Bellone, O. Maennel, and E. Malayjerdi. Analysing adversarial threats to rule-based local-planning algorithms for autonomous driving. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium*, 2023.
- [144] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [145] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine. Survivable key compromise in software update systems. In *Proceedings of the 17th ACM conference on Computer and communications security*, pages 61–72. ACM.
- [146] I. Z. Schlueter. kik, left-pad, and npm. <https://blog.npmjs.org/post/141577284765/kik-left-pad-and-npm>, Mar 2016.
- [147] J. Schlumberger, C. Kruegel, and G. Vigna. Jarhead analysis and detection of malicious java applets. In *Proceedings of the 28th Annual Computer Security Applications Conference, ACSAC '12*, page 249–257, New York, NY, USA, 2012. ACM.
- [148] T. R. Schorlemmer, K. G. Kalu, L. Chigges, K. M. Ko, E. A. Ishgair, S. Bagchi, S. Torres-Arias, and J. C. Davis. Signing in Four Public Software Package Registries: Quantity, Quality, and Influencing Factors. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1160–1178, Los Alamitos, CA, USA, May 2024. IEEE Computer Society.
- [149] I. Shani. Survey reveals AI's impact on the developer experience. <https://github.blog/2023-06-13-survey-reveals-ais-impact-on-the-developer-experience/>, Jun 2023.
- [150] Sigstore contributors. Sigstore. <https://docs.sigstore.dev/>.
- [151] Sigstore Contributors. Fulcio: Sigstore Certificate Authority. https://docs.sigstore.dev/certificate_authority/overview/, 2024. Accessed: 6 Jun 2025.
- [152] Sigstore Contributors. Rekor: Sigstore Transparency Log. <https://docs.sigstore.dev/logging/overview/>, 2024. Accessed: 6 Jun 2025.
- [153] Sigstore Contributors. Sigstore Security Model. <https://docs.sigstore.dev/about/security/>, 2024. Accessed: 6 Jun 2025.
- [154] SLSA contributors. SLSA Provenance.
- [155] R. J. Sofaer, Y. David, M. Kang, J. Yu, Y. Cao, J. Yang, and J. Nieh. Rogueone: Detecting Rogue Updates via Differential Data-Flow Analysis Using Trust Domains. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 1235–1247, Los Alamitos, CA, USA, Apr. 2024. IEEE Computer Society.
- [156] M. Souppaya, K. Scarfone, and D. Dodson. NIST Special Publication 800-218 - Secure Software Development Framework (SSDF) Version 1.1. NIST Computer Security Research Center Publications, Feb. 2022.
- [157] SPDX contributors. SPDX Overview. <https://spdx.dev/about/>.
- [158] C.-A. Staicu, S. Rahaman, Á. Kiss, and M. Backes. Bilingual problems: Studying the security risks incurred by native extensions in scripting languages. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6133–6150, Anaheim, CA, Aug. 2023. USENIX Association.
- [159] T. Stalaker, N. Wintersgill, O. Chaparro, M. D. Penta, D. German, and D. Poshyvanyk. Boms away! inside the minds of stakeholders: A comprehensive study of bills of materials for software systems. In *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pages 506–518, Los Alamitos, CA, USA, apr 2024. IEEE Computer Society.
- [160] J. Sun, J. Chen, Z. Xing, Q. Lu, X. Xu, and L. Zhu. Where is it? tracing the vulnerability-relevant files from vulnerability reports. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. ACM.
- [161] L. Sun. Using rekor-monitor to Scan Your Transparency Logs. <https://blog.sigstore.dev/using-rekor-monitor/>, retrieved June 2025, 2024.
- [162] Synopsys, Inc. Coverity Static Application Security Testing. <https://www.synopsys.com/software-integrity/security-testing/static-analysis-sast.html>.
- [163] tedhudek. Authenticode digital signatures - windows drivers.

- <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/authenticode>.
- [164] N. Telecommunications and I. Administration. Software Bill of Materials. <https://www.ntia.gov/SBOM>, 2021.
 - [165] The White House. Executive Order on Improving the Nation's Cybersecurity. <https://www.nist.gov/itl/executive-order-14028-improving-nations-cybersecurity>, May 2021.
 - [166] E. Thompson. NPM Docs: npm publish. <https://docs.npmjs.com/cli/v6/commands/npm-publish>.
 - [167] E. Topalovic, B. Saeta, L.-S. Huang, C. Jackson, and D. Boneh. Towards short-lived certificates. In *Proc. of Web 2.0 Security & Privacy (W2SP) '12*, 2012.
 - [168] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410, Santa Clara, CA, Aug. 2019. USENIX Association.
 - [169] S. Torres-Arias, A. K. Ammula, R. Curtmola, and J. Cappos. On omitting commits and committing omissions: Preventing git metadata tampering that (re) introduces software vulnerabilities. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 379–395, 2016.
 - [170] TruffleSecurity Co. Trufflehog. <https://github.com/trufflesecurity/trufflehog>.
 - [171] D. Uberti, J. Rundle, and C. Stupp. The Log4j Vulnerability: Millions of Attempts Made Per Hour to Exploit Software Flaw. <https://www.wsj.com/articles/what-is-the-log4j-vulnerability-11639446180>, Dec. 2021.
 - [172] Ubuntu Community. Apparmor. <https://wiki.ubuntu.com/AppArmor>, 2025. Ubuntu Wiki, accessed 6 Jun 2025.
 - [173] R. Uhlig, G. Neiger, D. Rodgers, A. Santoni, F. Martins, A. Anderson, S. Bennett, A. Kagi, F. Leung, and L. Smith. Intel virtualization technology. *Computer*, 38(5):48–56, 2005.
 - [174] N. Vasilakis, A. Benetopoulos, S. Handa, A. Schoen, J. Shen, and M. C. Rinard. Supply-chain vulnerability elimination via active learning and regeneration. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1755–1770. ACM.
 - [175] D. L. Vu, I. Pashchenko, F. Massacci, H. Plate, and A. Sabetta. Towards Using Source Code Repositories to Identify Software Supply Chain Attacks. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 2093–2095, Virtual Event USA, Oct. 2020. ACM.
 - [176] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham. Efficient software-based fault isolation. In *Proceedings of the fourteenth ACM symposium on Operating systems principles*, pages 203–216, 1993.
 - [177] F. Wang, C. Yun, S. Goldwasser, V. Vaikuntanathan, and M. Zaharia. Splinter: Practical private queries on public data. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 299–313, Boston, MA, Mar. 2017. USENIX Association.
 - [178] P. Wang, Y. Ding, M. Sun, H. Wang, T. Li, R. Zhou, Z. Chen, and Y. Jing. Building and maintaining a third-party library supply chain for productive and secure sgx enclave development. In *2020 IEEE/ACM 42nd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 100–109, 2020.
 - [179] W. Wang, X. Lin, J. Wang, W. Gao, D. Gu, W. Lv, and J. Wang. HODOR: Shrinking Attack Surface on Node.js via System Call Limitation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2023.
 - [180] X. Wang, Y. Zhang, X. Wang, Y. Jia, and L. Xing. Union under duress: Understanding hazards of duplicate resource mismediation in android software supply chain. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3403–3420, Anaheim, CA, Aug. 2023. USENIX Association.
 - [181] H. Wen, P. Porras, V. Yegneswaran, A. Gehani, and Z. Lin. 5G-Spector: An O-RAN Compliant Layer-3 Cellular Attack Detection Service. In *Proceedings 2024 Network and Distributed System Security Symposium*, San Diego, CA, USA, 2024. Internet Society.
 - [182] D. Wermke, J. H. Klemmer, N. Wöhler, J. Schmöser, H. S. Ramulu, Y. Acar, and S. Fahl. "always contribute back": A qualitative study on security challenges of the open source supply chain. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 1545–1560, 2023.
 - [183] D. Wermke, N. Wöhler, J. H. Klemmer, M. Fourné, Y. Acar, and S. Fahl. Committed to trust: A qualitative study on security trust in open source software projects. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1880–1896, 2022.
 - [184] C. Wohlin. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, EASE '14*, pages 1–10, New York, NY, USA, May 2014. ACM.
 - [185] S. Woo, E. Choi, H. Lee, and H. Oh. V1SCAN: Discovering 1-day vulnerabilities in reused C/C++ open-source software components using code classification techniques. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6541–6556, Anaheim, CA, 2023. USENIX Association.
 - [186] A. Wool. The use and usability of direction-based filtering in firewalls. *Computers and Security*, 23(6):459–468.
 - [187] C. Wright. Chapter 20 - risk management, security compliance, and audit controls. In C. Wright, editor, *The IT Regulatory and Standards Compliance Handbook*, pages 577–607. Syngress, Burlington, 2008.
 - [188] Y. Wu, J. Wang, Y. Wang, S. Zhai, Z. Li, Y. He, K. Sun, Q. Li, and N. Zhang. Your firmware has arrived: A study of firmware update vulnerabilities. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5627–5644, Philadelphia, PA, Aug. 2024. USENIX Association.
 - [189] E. Wyss, A. Wittman, D. Davidson, and L. De Carli. Wolf at the Door: Preventing Install-Time Attacks in npm with Latch. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pages 1139–1153, Nagasaki Japan, May 2022. ACM.
 - [190] B. Xia, T. Bi, Z. Xing, Q. Lu, and L. Zhu. An empirical study on software bill of materials: Where we stand and the road ahead. In *Proceedings of the 45th International Conference on Software Engineering, ICSE '23*, page 2630–2642. IEEE Press, 2023.
 - [191] J. Xiong, Y. Shi, B. Chen, F. R. Cogo, and Z. M. J. Jiang. Towards build verifiability for java-based systems. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP '22*, page 297–306, New York, NY, USA, 2022. ACM.
 - [192] D. Xue, G. Zhao, Z. Fan, W. Li, Y. Xu, Z. Liu, Y. Liu, and Z. Yuan. Poster: An Exploration of Large Language Models in Malicious Source Code Detection. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS '24*, pages 4940–4942. ACM, Dec. 2024.
 - [193] S. D. Young. Enhancing the Security of the Software Supply Chain through Secure Software Development Practices. <https://bidenwhitehouse.archives.gov/wp-content/uploads/2022/09/M-22-18.pdf>, Sep 2022.
 - [194] C. Yu, Y. Xiao, J. Lu, Y. Li, Y. Li, L. Li, Y. Dong, J. Wang, J. Shi, D. Bo, and W. Huo. File hijacking vulnerability: The elephant in the room. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium*, 2024.
 - [195] Yubico. YubiKey 5 Series. <https://www.yubico.com/products/yubikey-5-overview/>, retrieved June 2025, 2025.
 - [196] N. Zahan, P. Burckhardt, M. Lysenko, F. Aboukhadijeh, and L. Williams. Malwarebench: Malware samples are not enough. In *Proceedings of the 21st International Conference on Mining Software Repositories, MSR '24*, page 728–732, New York, NY, USA, 2024. ACM.
 - [197] N. Zahan, S. Shohan, D. Harris, and L. Williams. Do software security practices yield fewer vulnerabilities? In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, May 2023.
 - [198] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Madhila, and L. Williams. What Are Weak Links in the Npm Supply Chain? In *Proc. ACM ICSE-SEIP '22*, May 2022.
 - [199] J. Zhou. Top cyber security conferences ranking. <http://jianying.space/conference-ranking.html>. Accessed: 2025-08-06.
 - [200] J. Zhou, Z. Zhang, L. Ying, H. Chai, J. Cao, and H. Duan. Hey, Your Secrets Leaked! Detecting and Characterizing Secret Leakage in the Wild. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 449–467, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.
 - [201] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel. Small

Appendix A.

Generality and Fidelity of AStRA

To demonstrate that AStRA is not merely a theoretical exercise but a necessary framework for defense, we validate its Generality (its ability to model diverse ecosystems) and its Fidelity (its ability to explain real-world attacks).

A.1. Generality

We evaluate the generality of our model using two case studies widely studied in both scientific and gray literature: Left-pad (npm) [127] and SolarWinds [55].

Left-pad is a single-developer npm workflow, we determined the supply chain for it by analyzing its source code repository [4]. Succinctly put, the left-pad is a linear source code repository that, when the main developer deems appropriate, uses git tags to mark the release of a version. No automated pipeline is used to release, suggesting that the developer uses a script (i.e., [166] to upload the corresponding version to NPM. The same principal is in charge of pushing the resulting binary to a registry for consumers to download. Figure 3 represents the AStRA topology for Left-Pad.

SolarWinds is a multi-principal Tekton-based CI/CD pipeline that uses a combination of hosted services (e.g., GitHub) as well as self-hosted tools and infrastructure (e.g., Tekton) to provide a container build pipeline. Of particular interest is the use of different principals to report dependencies as a separate step. This is done this way because the automated pipeline pre-fetches the sources and installs them on the build infrastructure at the time of build. One possible reason for this is that developers need clearance and oversight when selecting third-party components. Separately, developers build their own source code and host it in a social coding platform, which is then tagged and released for build by the Tekton pipeline manager. The same principal is in charge of pushing the resulting binary to a registry for consumers to download. Figure 4 represents the AStRA topology for SolarWinds.

Debian's Reproducible Builds Large software ecosystems naturally produce sizable AStRA graphs, motivating dedicated tooling for their construction and analysis. To evaluate AStRA model practicality at scale, we have implemented a prototype that automatically generates AStRA graphs from Debian's reproducible-builds metadata [7], which covers more than a decade of Debian packaging activities. The resulting graph contains approximately 9 million nodes (≈ 7 M artifacts, 285 K resources, 1.3 M steps, and 1.2 K principals) and 407 M edges, with an average of ≈ 302 edges per step. These aggregate statistics confirm that AStRA can realistically represent production-scale supply chains and that such graphs are amenable to automated processing using existing graph-analysis frameworks (e.g.,

NetworkX, Neo4j). To show generality, we present an automated AStRA representation of a Debian buildserver using buildinfo [7] metadata on Figure 5.

Modeling SolarWinds, Left-pad, and Debian builds supply chains using the same four-element abstraction (Principals, Steps, Resources, and Artifacts) and without altering the schema, demonstrates that AStRA generalizes across manual and automated, single- and multi-actor workflows.

A.2. Fidelity

To evaluate this property, and given that we have proved the generality of our model, we will use it to analyze widely-known supply chain attacks on both SolarWinds and left-pad supply chains. We show how each compromise materializes within an AStRA model. **Left-Pad** was un-published by its author, as part of a dispute over a package name with the administrators of the npm package manager [146]. Since this was done without warning to the developers of the projects that depended on it, this affected many thousands of projects and npm observed hundreds of build failures per minute for the dependent projects and their dependents [146]. Based on our conceptual model, the artifact removal imposes a topological risk on all the software supply chain topologies that used left-pad as a direct or transitive dependency. This topology risk can be reduced using resource replication techniques to prevent a dependency from being a single point of failure in the build steps.

Open source package managers can benefit from analyzing the software supply chain topologies of the software they publish, to prevent incidents such as Left-Pad. Packages with high out-degrees can represent single point of failure, while packages with long dependency chains increase transitive risk.

SolarWinds In 2019, a sophisticated adversary used the SolarWinds Orion platform to plant stealthy backdoors in the networks of thousands of US government agencies and companies around the world [9, 141]. Although “SolarWinds” is used to refer to a single compromise, it was a campaign with multiple stages. The attackers injected malicious code known as Sunburst (aka Solorigate) into Orion during the build process. The report describes the reuse of credentials on the principals who carry out the build process. In this case, the nodes affected in Figure 4 were the CI/CD controller principals.

Further, the properties of the network structure can improve the analysis of assets and elements involved in a supply chain. For example, identifying principals with high out-degree would suggest an excess of privilege (and thus a higher compromise surface). Similar analysis on out-degree can be used to indicate artifact, resource, or even step criticality.

In both attacks, AStRA correctly localizes the source of the compromise (credential misuse or single-point dependency) and can trace its transitive effects across the supply chain.

This validation is critical for the systematization of this paper, as this allows us to confirm that Principals,

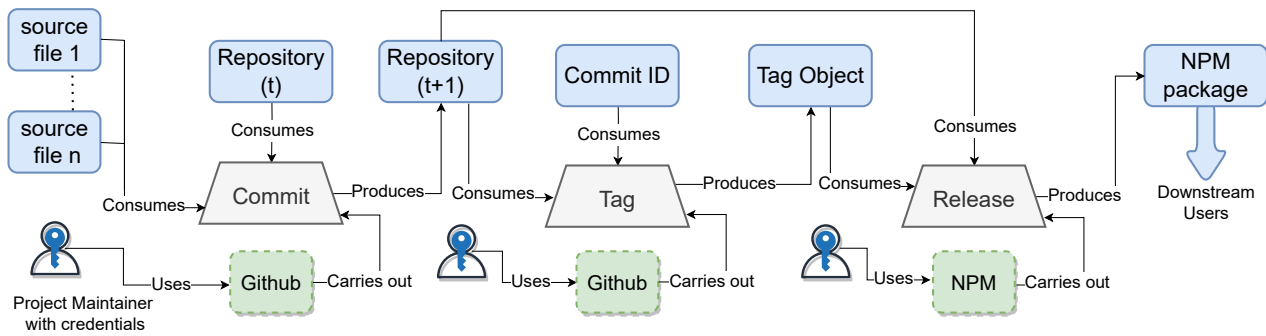


Figure 3. The supply chain DAG of left-pad, the commit step can occur multiple times before tagging the repo, and all the steps in figure occur for each release.

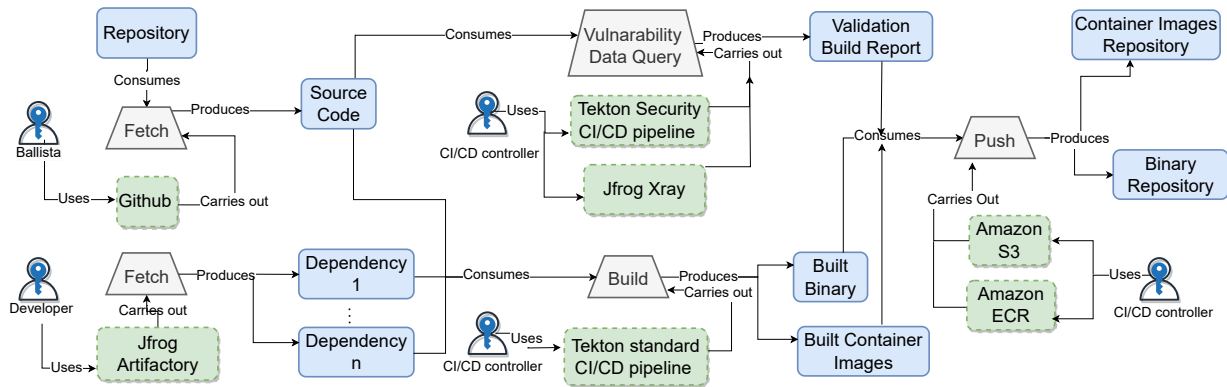


Figure 4. The supply chain DAG of SolarWinds build system.

Artifacts, Resources, Steps, and the DAG Structure create a model for defense. Consequently, the taxonomy we present is explicitly structured around these five elements. By securing these five objectives, we can mathematically assert coverage over the known classes of supply chain attacks.

Appendix B. SoK Methodology (Complements \$2)

From Phase 1 of our systematization, we identified 671 potential software supply chain security papers. Table B summarizes the venues represented in this corpus and lists each conference along with the number of papers contributed by that venue.

Fig. 6 shows the yearly distribution of references categorized by their primary focus, highlighting trends in academic attention within the software supply chain security field. We can see an overall growth in this area, with initial papers focusing on defenses, while most of the papers in recent years tend to do analysis (e.g., measurement studies of the state of security in the supply chain ecosystem). This is representative of a mature ecosystem, where researchers can measure the adoption of security practices. Consistent with this maturity, we can see that surveys and SoKs started in 2023. As mentioned above, a previous SoK focused on attacks [98], and a more recent one [74] focuses only on docker attacks and defenses. Our goal is to complement

TABLE 3. CONFERENCES USED TO SEED OUR SEARCH.

Conference	Paper Count
IEEE Symposium on Security and Privacy (US & Europe)	97
USENIX Security Symposium	137
ACM Conference on Computer and Communications Security (US & Asia)	138
Network and Distributed System Security Symposium	10
Annual Computer Security Applications Conference	33
IEEE Computer Security Foundations Symposium	8
European Symposium on Research in Computer Security	35
Privacy Enhancing Technologies Symposium	16
International Symposium on Research in Attacks, Intrusions and Defenses	16
Detection of Intrusions and Malware, and Vulnerability Assessment	6
International Conference on Software Engineering	109
Software Engineering in Practice (ICSE track)	12
Mining Software Repositories	26
USENIX Annual Technical Conference	8
Networked Systems Design and Implementation	12
Operating Systems Design and Implementation	8
Total	671

these previous efforts by focusing on defenses against a wide range of supply chain threats.

Table 4 contains 6 well-known papers from industry literature, and 24 tools/frameworks that were added to our corpus of academic literature.

Appendix C. Taxonomy (Complements \$4)

We present a version of our proposed software supply chain defenses taxonomy that lists all 76 papers and

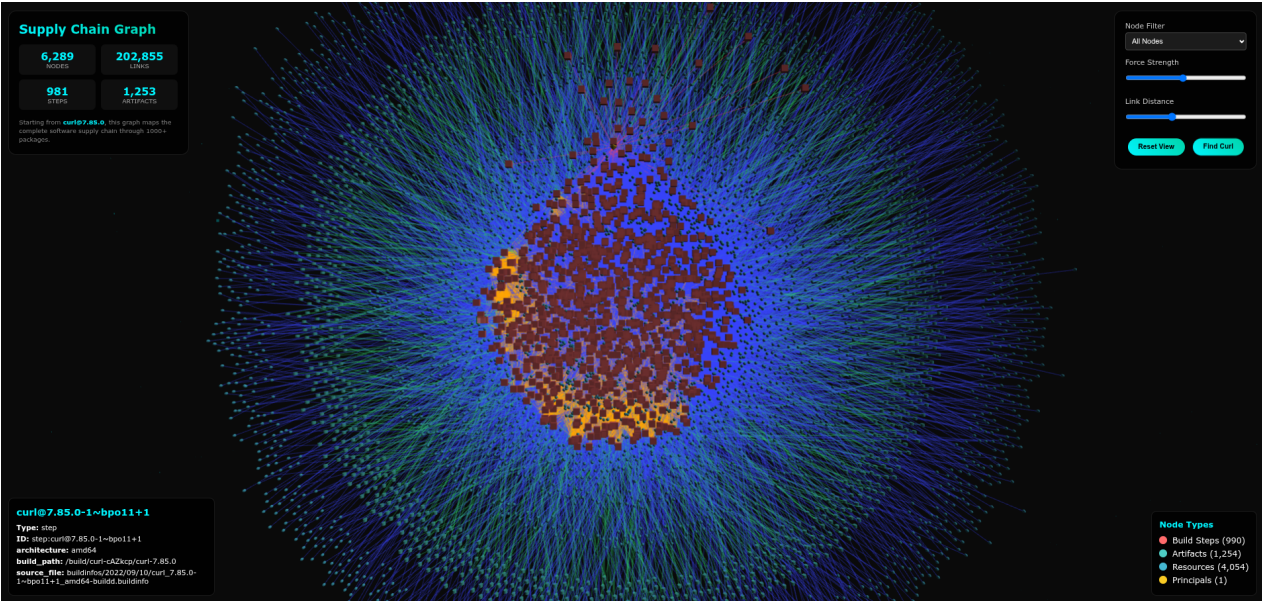


Figure 5. An automated supply chain representation for 990 debian builds.

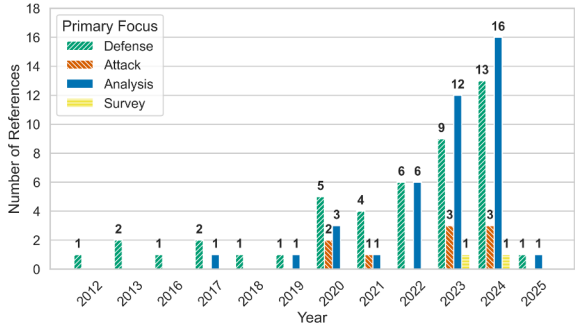


Figure 6. Reference counts by year and primary focus category (Defense, Attack, Security Analysis, Survey) in the software supply chain literature.

industry sources we systematized in table 5.

TABLE 4. DEFENSE WORKS EXTRACTED FROM GRAY LITERATURE AND INDUSTRY TOOLS.

Name	Reference	Type
SCITT (IETF)	[26]	White Paper
NIST SP 800-204D	[33]	White Paper
CIS Software Supply Chain Security Guide	[114]	White Paper
Linux Handbook: The Software Release Life Cycle in DevOps: Why Reinvent The Model?	[25]	White Paper
CNCF: Software Supply Chain Best Practices	[38]	White Paper
PEP 480 – End-to-end signing of packages	[96]	White Paper
Fulcio	[151]	Tool
Certificate Transparency	[1]	Tool
GitHub repository roles	[62]	Tool
NPM roles	[126]	Tool
Yubico (hardware MFA)	[195]	Tool
Git commit signing (GPG)	[60]	Tool
GitHub commit signing	[61]	Tool
Maven Maven GPG plugin	[56]	Tool
Docker Content Trust	[47]	Tool
Binary Transparency on Firefox	[5]	Tool
Sigstore	[150]	Tool
Scorecards	[133]	Tool
Coverity static analysis	[162]	Tool
AppArmor	[172]	Tool
SELinux	[8]	Tool
eBPF filtering	[52]	Tool
AMD64 Hardware virtualization	[13]	Tool
Intel Virtualization Technology	[173]	Tool
in-toto	[81]	Tool
Trivy	[24]	Tool
Supply-chain Levels for Software Artifacts (SLSA)	[154]	Framework
Software Bill of Materials (SBOM)	[164]	Framework
System Package Data Exchange (SPDX)	[157]	Framework
System Package Data Exchange (SPDX)	[157]	Framework

TABLE 5. SUPPLY CHAIN DEFENSE TECHNIQUES MAPPED TO SECURITY OBJECTIVES AND DEFENSE PATTERNS. ● DENOTES A TECHNIQUE THAT MEETS AN OBJECTIVE AND FALLS UNDER A DEFENSE PATTERN. ◐ DENOTES CONDITIONAL/LIMITED FULFILLMENT. △ DENOTES A POTENTIAL CONFLICT WITH OTHER OBJECTIVES. ✓ DENOTES A DEFENSE TECHNIQUE THAT WAS DEVELOPED SPECIFICALLY FOR THE SOFTWARE SUPPLY CHAIN.

AStRA Element	Supply Chain Specific?	Academic References	Industry References	Security Objective	Defense Pattern					
					Detective		Corrective		Preventative	
				P1 P2 P3 P4	Transparency	Integrity	Graceful degra.	Redundancy	Limitation	Correctness
Principals ephemeral credentials secret scanning credential transparency role separation multi-factor authentication separation of privilege threshold authorization		[121] [200] [121] [92] [121, 168]	[151] [1] [62, 126] [195] [38]	P1 ● P2 ● P3 ● P4 ●	● ●	● ●	● ●	● ●	● ◐	
Artifacts hashing artifact signing metadata signing artifact transparency ledger-based systems static analysis patch propagation dynamic analysis ML-based analysis	 ✓ ✓ ✓ 	[175] [121] [88] [113, 121] [18] [51, 64, 77, 78, 84, 99, 100, 117, 147, 155, 185] [50, 106, 107] [30, 75, 77, 100, 128, 136, 155, 174, 181, 192] [65, 84, 85, 100, 122, 147, 189]	[56, 60, 61] [47] [5, 133, 150] [162]	A1 ◐ A2 ● A3 ●	● ◐ ● ●	● ● ● ●			◐ ● ● ●	
Resources mandatory access control filtering virtualization		 [14] [69]	[8, 38, 172] [52] [13, 173]	R1 ● R2 ● R3 ●			●		● ●	
Steps deterministic steps step transparency step consensus static analysis ML-based analysis	✓ ✓ ✓ 	[41, 119, 191] [168] [54, 123] [112, 188] [73]	[24, 81, 154, 157, 164] [26]	S1 ● S2 ● S3 ●	◐ ●	● ●			● ◐ ●	
Supply Chain Structure supply chain layout integrity supply chain reproducibility	✓ ✓	[168] [191]	[81]	T1 ● T2 ●		●				●